Turtle Graphics Project, Spring 2026

Tong Yi

Contents

1 CopyRight Warning

1. This is copyrighted materials; you are not allowed to upload to the Internet.
2. Our project is different from similar products in Internet.
   (a) Ask help only from teaching staff of this course.
   (b) Use solutions from artificial intelligence like ChatGPT or online tutoring websites like, but not limited to, chegg.com, violates academic integrity and is not allowed.

2 Example of Separating Declaration, Definition, and Test Code: FuelTank Class

For robust software engineering practices, especially in large projects, the standard C++ approach is to divide the code into three distinct files:

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

Source File (.cpp): Contains the definitions (implementation logic) of the class's member functions.

Test File (.cpp): Contains the main() function and the code dedicated to testing the functionality of the class.

This separation improves compile times (only changed implementation files need recompilation), enhances code organization, and facilitates code reuse across different parts of a project.

Code link: https://onlinegdb.com/MzXcjYT2S-.

In online compilers like onlinegdb, the primary source file containing the `main` function must be named `main.cpp`. When working on your local computer, you have more flexibility and can name the test file something more descriptive, such as `FuelTankTest.cpp`. Naming the file `FuelTankTest.cpp` is generally preferred over `TestFuelTank.cpp` because it keeps related files grouped together when sorted alphabetically. This is a common practice in many C++ development environments to improve project organization and scannability.

2.1 FuelTank.hpp

```
1 #ifndef FUEL_TANK_H
2 #define FUEL_TANK_H
```

```
class FuelTank {
public:
    FuelTank();
    FuelTank(int startLevel);
    int getGasLevel() const;
    void fillGas(int numGallons);
    void sendGas(int numGallons);

private:
    int currGasLevel;
};
#endif
```

2.2 FuelTank.cpp

```
#include "FuelTank.hpp"

FuelTank::FuelTank() {
   //int currGasLevel = 0; //wrong, with int before currGasLevel, the currGasLevel is a
      local variable
   //for default constructor,
   //that statement does not initialize data member currGasLevel
   currGasLevel = 0;
}

FuelTank::FuelTank(int startGasLevel) {
    if (startGasLevel >= 0) {
       currGasLevel = startGasLevel;
    }
    else {
       currGasLevel = 0;
    }
}

int FuelTank::getGasLevel() const {
    return currGasLevel;
}

void FuelTank::sendGas(int gallons) {
    if (currGasLevel >= gallons) {
       currGasLevel -= gallons;
    }
}

void FuelTank::fillGas(int gallons) {
```

```
    currGasLevel += gallons;
}
```

2.3 FuelTankTest.cpp

```
//Sample output:
//gas level of an empty fuel tank is 0
//add 5 gallons to tank
//send 3 gallons out from tank
//current gas level is 2

#include "FuelTank.hpp"
#include <iostream>

int main() {
    FuelTank tank(0);

    std::cout << "gas level of an empty fuel tank is "
         << tank.getGasLevel() << std::endl;

    std::cout << "add 5 gallons to tank" << std::endl;
    tank.fillGas(5);

    std::cout << "send 3 gallons out from tank" << std::endl;
    tank.sendGas(3);

    std::cout << "current gas level is ";
    std::cout << tank.getGasLevel() << std::endl;

    return 0;
}
```

To run the code,

```
g++ -o run FuelTank.cpp FuelTankTest.cpp
./run
```

3 Overview of Turtle Graphics Project

This project involves three classes: Floor, Turtle, and TurtleGraphics.

- Floor is a square-shape grid with characters '*' and space character ' '. There are operations like `clear`, `mark`, `at` to operate on the grid.
- Turtle lets a turtle lift up or put down a pen, turning left or right, and move.

- TurtleGraphics lets a turtle to draw character '*' on the floor by turning, moving, and lifting up or putting down a pen properly.

To run the project, we do the following.

1. Build a directory in your computer by entering the following command with return key in a terminal.

```
mkdir TurtleGraphics
```

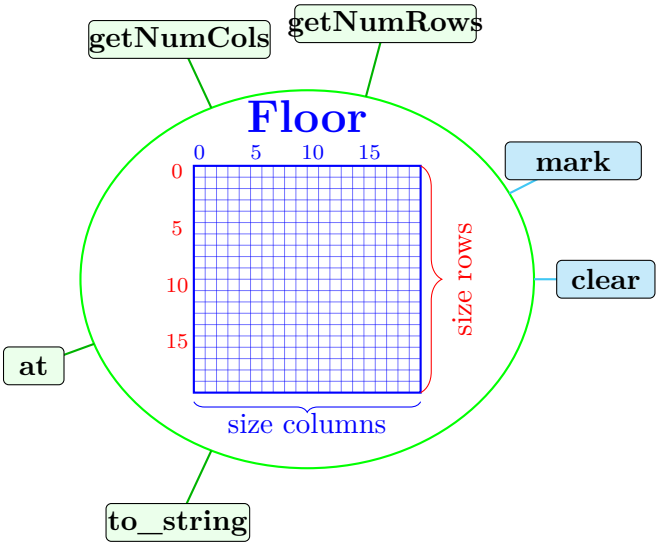
2. Move to the directory by pressing the following command with return key in a terminal.

```
cd TurtleGraphics
```

3. Download the required files and save them in the above directory.
4. You implement Floor, Turtle, and TurtleGraphics classes. Then test TurtleGraphics class. That is,
   (a) Define Floor.cpp in Task A.
   (b) Define Turtle.cpp in Task B.
   (c) Define TurtleGraphics.cpp in Task C.
   (d) Define drawSideBySide.cpp in Task D.

4 Task A: Implement Floor Class

4.1 Illustration of Floor Class



4.2 Download Floor.hpp

Download link: **Floor.hpp**. Its contents are as follows.

```cpp
#ifndef FLOOR_HPP
#define FLOOR_HPP

#include <vector>
#include <string>

/**
 * @class Floor
 * @brief Manages a 2D character grid with specific labeling and safety checks.
 */
class Floor {
public:
    /**
     * @brief Default constructor.
     * Initializes a default 20 x 20 grid filled with spaces (' ').
     */
    Floor();

    /**
     * @brief Parameterized constructor.
     * @param size The desired dimension for a square grid.
     * @note If size < 10, a default 20x20 grid is created instead.
     */
    Floor(int size);

    /** @return The number of rows in the grid. */
    int getNumRows() const;

    /** @return The number of columns in the grid. */
    int getNumCols() const;

    /**
     * @brief Places a character at the specified coordinates.
     * @param row The row index from 0 to getNumRows() - 1.
     * @param col The column index from 0 to getNumCols() - 1.
     * @param ch The character to place.
     * @note If indices are out of bounds, the operation is silently ignored.
     */
    void mark(int row, int col, char ch);

    /**
     * @brief Retrieves the character at the specified coordinates.
     * @param row The row index from 0 to getNumRows() - 1.
```

```
     * @param col The column index from 0 to getNumCols() - 1.
     * @return The character at (row, col), or the null character '\0' if out of bounds.
     */
    char at(int row, int col) const;

    /**
     * @brief Resets all cells in the grid to a space (' ').
     */
    void clear();

    /**
     * @brief Returns a string representation of the grid with coordinate labels.
     *
     * Labeling Rules:
     * 1. Designed for grids up to 99x99.
     * 2. The tens digit for row/column indices is displayed ONLY at multiples
     *    of 10 (e.g., 10, 20, 30), appearing directly above or to the left.
     * 3. The units digit is displayed for every row and column.
     *
     * @return A formatted string containing the labels and grid content.
     */
    std::string to_string() const;

private:
    std::vector<std::vector<char>> grid;
};

#endif
```

4.3 Implement Floor.cpp

Read the requirement of Floor.hpp. Implement the constructors and methods in Floor.cpp.

4.4 Test Floor.cpp using FloorUnitTest.cpp

Download link: **FloorUnitTest.cpp** – its contents are as follows. The purpose is to test the constructors and methods of Floor.cpp.

```
//File name: FloorUnitTest.cpp
#include <iostream>
#include <vector>
#include <cstdlib> //srand
#include <ctime> //time
#include <sstream> //std::ostringstream
#include "Floor.hpp"
//g++ -o floor Floor.cpp FloorUnitTest.cpp
```

```
//test default constructor using
//./floor A

const int SIZE = 16;
Floor assignData(char data[][SIZE]);
void printCanvas(const Floor& canvas, const std::string& prompt);

int main(int argc, const char *argv[]) {
    if (argc != 2) {
       std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
       return -1;
    }

    //unit-testing for constructors and methods of Floor class
    char type = *argv[1];
    std::string prompt;

    //IMPORTANT: If your output is inconsistent, temporarily replace srand(time(NULL))
       with srand(0).
    srand(time(NULL));

    //'A' for default constructor and
    //'B' for non-default constructor
    if (type == 'A' || type == 'B') {
      if (type == 'A') {
        prompt = "Call default constructor Floor canvas;";
        Floor canvas;
        printCanvas(canvas, prompt);
//Sample output:
//Call default constructor Floor canvas;
//num of rows: 20
//num of cols: 20
//contents of canvas is
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , ,
```

```
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,
// , , , , , , , , , , , , , , , , , , , , ,

      }
      else if (type == 'B') {
            std::cout << "Enter size of the canvas (needs to be at least 10, otherwise,
               convert it to 20): ";
            int size;
            std::cin >> size;
            prompt = "Call non-default constructor Floor canvas(" + std::to_string(size)
                + ");";
            Floor canvas(size);
            printCanvas(canvas, prompt);

//Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 11
//Call non-default constructor Floor canvas(11);
//num of rows: 11
//num of cols: 11
//contents of canvas is
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
// , , , , , , , , , , ,
//

//do not take the parameter as it is,
//need to make sure that parameter size is >= 10
//Sample input/output:
//Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 5
//Call non-default constructor Floor canvas(5);
//num of rows: 20
//num of cols: 20
//contents of canvas is
// , , , , , , , , , , , , , , , , , , , , ,
```

```
98  // , , , , , , , , , , , , , , , , , , , , , , ,
99  // , , , , , , , , , , , , , , , , , , , , , , ,
100 // , , , , , , , , , , , , , , , , , , , , , , ,
101 // , , , , , , , , , , , , , , , , , , , , , , ,
102 // , , , , , , , , , , , , , , , , , , , , , , ,
103 // , , , , , , , , , , , , , , , , , , , , , , ,
104 // , , , , , , , , , , , , , , , , , , , , , , ,
105 // , , , , , , , , , , , , , , , , , , , , , , ,
106 // , , , , , , , , , , , , , , , , , , , , , , ,
107 // , , , , , , , , , , , , , , , , , , , , , , ,
108 // , , , , , , , , , , , , , , , , , , , , , , ,
109 // , , , , , , , , , , , , , , , , , , , , , , ,
110 // , , , , , , , , , , , , , , , , , , , , , , ,
111 // , , , , , , , , , , , , , , , , , , , , , , ,
112 // , , , , , , , , , , , , , , , , , , , , , , ,
113 // , , , , , , , , , , , , , , , , , , , , , , ,
114 // , , , , , , , , , , , , , , , , , , , , , , ,
115 // , , , , , , , , , , , , , , , , , , , , , , ,
116 // , , , , , , , , , , , , , , , , , , , , , , ,
117 //
118       }
119     }
120     else if (type >= 'C' && type <= 'E') {
121         char data[SIZE][SIZE];
122
123         //set some random values for data
124         char chosenValues[] = {'*', ' '};
125         int numChosenValues = sizeof(chosenValues) / sizeof(chosenValues[0]);
126         for (int row = 0; row < SIZE; row++) {
127           for (int col = 0; col < SIZE; col++) {
128             data[row][col] = chosenValues[rand() % numChosenValues];
129           }
130         }
131         Floor canvas = assignData(data);
132
133         if (type == 'C') {
134            //test clear method,
135            canvas.clear();
136
137            //after calling clearing method, each element of board should be ' '
138            bool isWrong = false;
139            for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
140              for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
141                if (canvas.at(row, col) != ' ') {
142                  std::cout << "clear method is not correct. Each element should be a
                       space character\n";
```

```
143                 isWrong = true; //set a tag
144                 //break; //break only can break the inner loop, the outer loop still
                        runs
145               }
146             }
147           }
148
149           if (!isWrong) {
150             std::cout << "clear method is correct\n";
151           }
152           else {
153             std::cout << "clear method is not correct\n";
154           }
155         //expected output:
156         //clear method is correct
157         }
158         else if (type == 'D') {
159           //test at method
160           bool isWrong = false;
161           for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
162               for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
163                   if (canvas.at(row, col) != data[row][col]) {
164                     std::cout << "at method is not correct. The expected value is '" <<
                              data[row][col] << "'"
165                            << ", however, the actual value is '" << canvas.at(row, col) << "
                                 '\n";
166                     isWrong = true; //set a tag
167                     //break; //break only can break the inner loop, the outer loop
                            still runs
168                   }
169               }
170           }
171
172           if (!isWrong) {
173              std::cout << "at method is correct\n";
174           }
175           else {
176             std::cout << "at method is not correct\n";
177           }
178         }
179         else if (type == 'E') {
180               //canvas.print(); //debug purpose
181
182               // build expected output manually from data[][]
183               std::ostringstream expected;
184
```

```
            // print actual via stream
            std::string actual = canvas.to_string();

            // build expected string from data[][]
            expected << " ";
            for (int col = 0; col < SIZE; col++) {
              if (col % 10 == 0 && col / 10 >= 1) {
                expected << col / 10;
              }
              else {
                expected << ' ';
              }
            }
            expected << '\n';

            expected << " ";
            for (int col = 0; col < SIZE; col++) {
              expected << col % 10;
            }
            expected << '\n';

            for (int row = 0; row < SIZE; row++) {
              expected << (row % 10 == 0 && row / 10 >= 1 ? char('0' + row / 10) : ' '
                  );
              expected << row % 10;
              for (int col = 0; col < SIZE; col++) {
                expected << data[row][col];
              }
              expected << '\n';
            }

            //std::cout << "'" << actual << "'\n\n";
            //std::cout << "'" << expected.str() << "'\n";
            if (actual == expected.str()) {
              std::cout << "to_string method is correct\n";
            }
            else {
              std::cout << "to_string method is not correct\n";
            }
         //expected output:
         //to_string method is correct
        }
    }
    else if (type == 'F') {
      //test mark method of Floor class
      Floor canvas(SIZE);
```

```
230       canvas.mark(0, 0, '*');        // top-left corner
231       canvas.mark(SIZE-1, SIZE-1, '*');       // bottom-right corner, minimum size is 10
232       bool cornersCorrect = canvas.at(0,0) == '*' && canvas.at(SIZE-1, SIZE-1) == '*';
233
234       canvas.mark(-1, 0, '*');    // out of bounds, should do nothing
235       canvas.mark(0, SIZE, '*');  // out of bounds, should do nothing
236       canvas.mark(SIZE, 0, '*');  // out of bounds, should do nothing
237       canvas.mark(SIZE, SIZE, '*'); // out of bounds, should do nothing
238
239       // at() should return '\0' for invalid indices
240       bool outOfBoundsCorrect = canvas.at(-1,0) == '\0' &&
241         canvas.at(0,SIZE) == '\0' && canvas.at(SIZE,0) == '\0' && canvas.at(SIZE,SIZE)
              == '\0';
242
243       int row = SIZE / 2;
244       int col = SIZE / 2;
245       canvas.mark(SIZE/2, SIZE/2, '*');
246       bool centerCorrect = canvas.at(row, col) == '*';
247
248       row = rand() % SIZE;
249       col = rand() % SIZE;
250       canvas.mark(row, col, '*'); // ok even if (row,col) overlaps (0,0) or (SIZE-1,
            SIZE-1) since they are already '*'
251
252
253       bool randomCorrect = canvas.at(row, col) == '*';
254       if (cornersCorrect && outOfBoundsCorrect && centerCorrect && randomCorrect) {
255         std::cout << "mark method is correct\n";
256       }
257       else {
258         std::cout << "mark method is not correct\n";
259       }
260
261 //sample output:
262 //mark method is correct
263     }
264     else {
265       std::cout << "Unrecognized option: " << type << std::endl;
266       return -1;
267     }
268
269     return 0;
270 }
271
272 Floor assignData(char data[][SIZE]) {
273    Floor canvas(SIZE);
```

```
274
275     for (int row = 0; row < canvas.getNumRows(); row++) {
276         for (int col = 0; col < canvas.getNumCols(); col++) {
277             canvas.mark(row, col, data[row][col]); //set board[row][col] of *canvasPtr --
                    a Floor object -- to be data[row][col]
278         }
279     }
280
281     return canvas;
282 }
283
284 void printCanvas(const Floor& canvas, const std::string& prompt) {
285     std::cout << prompt << '\n';
286     std::cout << "num of rows: " << canvas.getNumRows() << '\n';
287     std::cout << "num of cols: " << canvas.getNumCols() << '\n';
288     std::cout << "contents of canvas is\n";
289     for (int row = 0; row < canvas.getNumRows(); row++) {
290         for (int col = 0; col < canvas.getNumCols(); col++) {
291             std::cout << canvas.at(row, col) << ',';
292         }
293         std::cout << '\n';
294     }
295 }
```

Run the following commands.

```
1 g++ -o floor Floor.cpp FloorUnitTest.cpp
```

If the code compiles, you would get a `floor` executable file. To test the default constructor, run

```
1 ./floor A
```

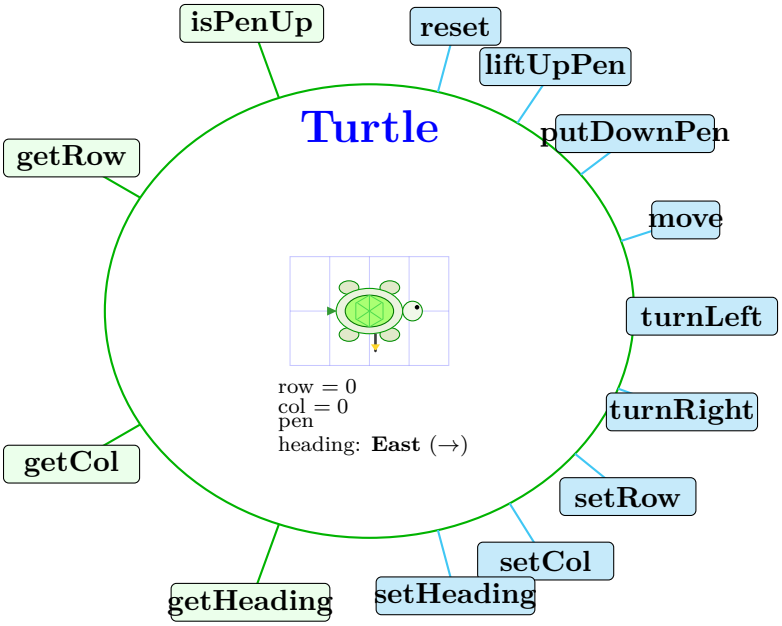
Test the rest of constructors and methods according to the comments in `FloorUnitTest.cpp`.

4.5 Submission for Task A

Submit `Floor.cpp` to gradescope.

5 Task B: Implement Turtle Class

5.1 Illustration of Turtle Class



5.2 Download Turtle.hpp

Download link: **Turtle.hpp** Its contents are as follows.

```cpp
// File name: Turtle.hpp
#ifndef Turtle_HPP
#define Turtle_HPP

#include <ostream> //std::ostream, used in << operator for Direction

/**
 * @brief Represents the state of the turtle's drawing pen.
 *
 * UP  - the pen is lifted; movement does not draw.
 * DOWN - the pen is lowered; movement draws a trail.
 */
enum class Pen { UP, DOWN };

/**
 * @brief Represents the four cardinal directions the turtle can face.
 */
enum class Direction { NORTH, EAST, SOUTH, WEST };

/**
 * @brief Overloads the stream insertion operator for Direction.
 * @param os    The output stream to write to.
```

```
 * @param heading The Direction value to print.
 * @return       Reference to the output stream, with the direction written
 *               as a human-readable string: "NORTH", "EAST", "SOUTH", or "WEST".
 */
std::ostream& operator<<(std::ostream& os, Direction heading);

/**
 * @brief Models a turtle that moves on a 2-D grid and optionally draws a trail.
 *
 * The turtle tracks its position (row, col), the direction it faces (heading),
 * and whether its pen is up or down. All data members are initialized at the
 * point of declaration so no user-defined constructor is required.
 */
class Turtle {
public:

  // No user-defined constructor is needed.
  // All data members are initialized where they are declared (modern C++ style),
  // and the compiler-generated default constructor is sufficient.

  /**
   * @brief Resets the turtle to its initial state.
   *
   * After this call:
   *   - pen    == Pen::UP
   *   - heading == Direction::EAST
   *   - row    == 0
   *   - col    == 0
   */
  void reset();

  /**
   * @brief Reports whether the pen is currently lifted.
   * @return true if pen == Pen::UP (turtle does not draw while moving).
   * @return false if pen == Pen::DOWN (turtle draws while moving).
   */
  bool isPenUp() const;

  /**
   * @brief Lifts the pen so that subsequent moves do not draw.
   * @post pen == Pen::UP
   */
  void liftUpPen();

  /**
   * @brief Lowers the pen so that subsequent moves draw a trail.
```

```
     * @post pen == Pen::DOWN
     */
    void putDownPen();

    /**
     * @brief Moves the turtle forward by the given number of steps in its current heading
        .
     *
     * The axis updated depends on the current heading:
     *   - EAST  : col += steps
     *   - WEST  : col -= steps
     *   - SOUTH : row += steps
     *   - NORTH : row -= steps
     *
     * Negative values for steps move the turtle in the opposite direction.
     * No bounds checking is performed; the caller is responsible for
     * keeping the turtle within any grid boundaries.
     *
     * @param steps Number of grid cells to move (may be negative).
     */
    void move(int steps);

    /**
     * @brief Rotates the turtle 90 degrees to the left (counter-clockwise).
     *
     * Heading transitions:
     *   EAST  -> NORTH
     *   NORTH -> WEST
     *   WEST  -> SOUTH
     *   SOUTH -> EAST
     *
     * @post heading is updated to the next counter-clockwise direction.
     */
    void turnLeft();

    /**
     * @brief Rotates the turtle 90 degrees to the right (clockwise).
     *
     * Heading transitions:
     *   EAST  -> SOUTH
     *   SOUTH -> WEST
     *   WEST  -> NORTH
     *   NORTH -> EAST
     *
     * @post heading is updated to the next clockwise direction.
     */
```

```
114     void turnRight();
115
116     /**
117      * @brief Sets the turtle's row position on the grid.
118      *
119      * No bounds checking is performed; the caller is responsible for
120      * passing a valid row index.
121      *
122      * @param row The new row index (0-based).
123      */
124     void setRow(int row);
125
126     /**
127      * @brief Sets the turtle's column position on the grid.
128      *
129      * No bounds checking is performed; the caller is responsible for
130      * passing a valid column index.
131      *
132      * @param col The new column index (0-based).
133      */
134     void setCol(int col);
135
136     /**
137      * @brief Returns the turtle's current row position.
138      * @return The current row index (0-based).
139      */
140     int getRow() const;
141
142     /**
143      * @brief Returns the turtle's current column position.
144      * @return The current column index (0-based).
145      */
146     int getCol() const;
147
148     /**
149      * @brief Sets the direction the turtle is facing.
150      *
151      * Because Direction is a scoped enum, only the four defined values
152      * (NORTH, EAST, SOUTH, WEST) are valid; no explicit validation is needed.
153      *
154      * @param heading The new heading for the turtle.
155      */
156     void setHeading(Direction heading);
157
158     /**
159      * @brief Returns the direction the turtle is currently facing.
```

```
   * @return The current heading as a Direction value.
   */
  Direction getHeading() const;

private:
  // Modern C++ preferred style: initialize members where they are declared,
  // and let the compiler generate the default constructor.
  Pen       pen     = Pen::UP;         ///< Current pen state (UP or DOWN).
  Direction heading = Direction::EAST; ///< Current heading (NORTH/EAST/SOUTH/WEST).
  int       row     = 0;               ///< Current row position on the grid (0-based).
  int       col     = 0;               ///< Current column position on the grid (0-based)
      .
};

#endif
```

5.3 Implement Turtle.cpp

Read the requirements in `Turtle.hpp`. Implement all methods declared in `Turtle.hpp` inside `Turtle.cpp`. For your convience, here are the code to overload >> operator for enum Direction class.

```
std::ostream& operator<<(std::ostream& os, Direction heading) {
    switch (heading) {
        case Direction::EAST: return os << "EAST";
        case Direction::WEST: return os << "WEST";
        case Direction::SOUTH: return os << "SOUTH";
        case Direction::NORTH: return os << "NORTH";
        default:          return os << "Unknown";
    }
}
```

5.4 Test Turtle.cpp using TurtleUnitTest.cpp

Download link: **TurtleUnitTest.cpp** Its contents are as follows.

```
//file name: TurtleUnitTest.cpp
#include <iostream>
#include <cstdlib> //srand
#include <ctime> //time
#include "Turtle.hpp"
//g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
//test default constructor using
//./turtle A

int main(int argc, const char *argv[]) {
  if (argc != 2) {
```

```
12      std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
13      return -1;
14    }
15
16    //unit-testing for methods of Turtle class, which has no constructors
17    char type = *argv[1];
18    std::string prompt;
19
20    srand(time(NULL));
21    const int SIZE = 20;
22    switch (type) {
23      case 'A': { //test reset method
24        Turtle tur;
25        tur.reset();
26        if (tur.isPenUp() && tur.getRow() == 0 && tur.getCol() == 0 && tur.getHeading() ==
              Direction::EAST) {
27          std::cout << "reset method is correct" << std::endl;
28        }
29        else {
30          std::cout << "reset method is not correct" << std::endl;
31        }
32        break;
33      }
34  //When running ./turtle A,
35  //where turtle obtained by
36  //g++ -o turtle TurtleUnitTest.cpp Turtle.cpp
37  //expected output is as follows
38  //reset method is correct
39
40
41      case 'B': { //test isPenUp, liftUpPen, and putDownPen methods
42        Turtle tur;
43        tur.liftUpPen();
44        bool penUpTrue = tur.isPenUp();
45
46        tur.putDownPen();
47        bool penUpFalse = tur.isPenUp();
48
49        if (penUpTrue && penUpFalse == false) {
50          std::cout << "isPenUp, liftUpPen, and putDownPen methods are correct" << std::
                endl;
51        }
52        else {
53          std::cout << "isPenUp, liftUpPen, and putDownPen methods are not correct" << std
                ::endl;
54        }
```

```
55
56        break;
57      }
58  //Running the following two comands
59  //g++ -o turtle TurtleUnitTest.cpp Turtle.cpp
60  //./turtle B
61  //expected output: isPenUp, liftUpPen, and putDownPen methods are correct
62
63      case 'C': { //test setRow and getRow methods
64        Turtle tur;
65        int row = rand() % SIZE;
66        tur.setRow(row);
67        if (tur.getRow() == row) {
68          std::cout << "setRow and getRow methods are correct" << std::endl;
69        }
70        else {
71          std::cout << "setRow and getRow methods are not correct" << std::endl;
72        }
73
74        break;
75      }
76  //expected output after running ./turtle C
77  //setRow and getRow methods are correct
78
79      case 'D': { //test setCol and getCol methods
80        Turtle tur;
81        int col = rand() % SIZE;
82        tur.setCol(col);
83        if (tur.getCol() == col) {
84          std::cout << "setCol and getCol methods are correct" << std::endl;
85        }
86        else {
87          std::cout << "setCol and getCol methods are not correct" << std::endl;
88        }
89
90        break;
91      }
92  //expected output after running ./turtle D
93  //setCol and getCol methods are correct
94
95      case 'E': { //test turnLeft, setHeading and getHeading methods
96        Turtle tur;
97
98        tur.setHeading(Direction::EAST);
99        tur.turnLeft();
100       if (tur.getHeading() != Direction::NORTH) {
```

```
101         std::cout << "When direction is EAST, turnLeft method is not correct." << std::
                endl;
102         break;
103       }
104
105       tur.setHeading(Direction::WEST);
106       tur.turnLeft();
107       if (tur.getHeading() != Direction::SOUTH) {
108         std::cout << "When direction is WEST, turnLeft method is not correct." << std::
                endl;
109         break;
110       }
111
112       tur.setHeading(Direction::NORTH);
113       tur.turnLeft();
114       if (tur.getHeading() != Direction::WEST) {
115         std::cout << "When direction is NORTH, turnLeft method is not correct." << std::
                endl;
116         break;
117       }
118
119       tur.setHeading(Direction::SOUTH);
120       tur.turnLeft();
121       if (tur.getHeading() != Direction::EAST) {
122         std::cout << "When direction is SOUTH, turnLeft method is not correct." << std::
                endl;
123         break;
124       }
125
126       std::cout << "turnLeft method is correct." << std::endl;
127       break;
128     }
129 //expected output after running ./turtle E:
130 //turnLeft method is correct.
131
132     case 'F': { //test turnRight, setHeading and getHeading methods
133       Turtle tur;
134
135       tur.setHeading(Direction::EAST);
136       tur.turnRight();
137       if (tur.getHeading() != Direction::SOUTH) {
138         std::cout << "When direction is EAST, turnRight method is not correct." << std::
                endl;
139         break;
140       }
141
```

```
142       tur.setHeading(Direction::WEST);
143       tur.turnRight();
144       if (tur.getHeading() != Direction::NORTH) {
145         std::cout << "When direction is WEST, turnRight method is not correct." << std::
              endl;
146         break;
147       }
148
149       tur.setHeading(Direction::NORTH);
150       tur.turnRight();
151       if (tur.getHeading() != Direction::EAST) {
152         std::cout << "When direction is NORTH, turnRight method is not correct." << std
              ::endl;
153         break;
154       }
155
156       tur.setHeading(Direction::SOUTH);
157       tur.turnRight();
158       if (tur.getHeading() != Direction::WEST) {
159         std::cout << "When direction is SOUTH, turnRight method is not correct." << std
              ::endl;
160         break;
161       }
162
163       std::cout << "turnRight method is correct." << std::endl;
164       break;
165     }
166 //expected output after running ./turtle F:
167 //turnRight method is correct.
168
169     case 'G': { // void move(int steps);
170       Turtle tur;
171
172       //move in east direction
173       tur.setHeading(Direction::EAST);
174       int numStepsEast = 0;
175       while (numStepsEast == 0) {
176         numStepsEast = rand() % SIZE;
177       }
178       //Make sure numStepsEast is not zero.
179
180       tur.move(numStepsEast);
181
182       if (tur.getRow() != 0 || tur.getCol() != numStepsEast) {
183         std::cout << "When direction is Direction::EAST, move method of " <<
              numStepsEast << " steps is wrong." << std::endl;
```

```
184         break;
185       }
186
187       //move in south direction from (0, numStepsEast)
188       tur.setHeading(Direction::SOUTH);
189       int numStepsSouth = 0;
190       while (numStepsSouth == 0) {
191         numStepsSouth = rand() % SIZE;
192       }
193       //Make sure numStepsSouth is not zero.
194
195       tur.move(numStepsSouth);
196       if (tur.getRow() != numStepsSouth || tur.getCol() != numStepsEast) {
197         std::cout << "When direction is Direction::SOUTH, move method of " <<
                numStepsSouth << " steps is wrong." << std::endl;
198         break;
199       }
200
201       //set tur to the rightmost position of the first row, where row index is 0 and
            column index is SIZE-1.
202       tur.setRow(0);
203       tur.setCol(SIZE-1);
204
205       //move in west direction
206       tur.setHeading(Direction::WEST);
207
208       int numStepsWest = 0;
209       while (numStepsWest == 0) {
210         numStepsWest = rand() % SIZE;
211       }
212       //Make sure numStepsWest is not zero.
213
214       tur.move(numStepsWest);
215       if (tur.getRow() != 0 || tur.getCol() != SIZE -1 -numStepsWest) {
216         std::cout << "When direction is Direction::WEST, move method of " <<
                numStepsWest << " steps is wrong." << std::endl;
217         break;
218       }
219
220       //set tur to the bottom right position in the grid, where row index is SIZE-1 and
            column index is SIZE-1.
221       tur.setRow(SIZE-1);
222       tur.setCol(SIZE-1);
223
224       //move in north direction
225       tur.setHeading(Direction::NORTH);
```

```
      int numStepsNorth = 0;
      while (numStepsNorth == 0) {
        numStepsNorth = rand() % SIZE;
      }
      //Make sure numStepsNorth is not zero.

      tur.move(numStepsNorth);
      if (tur.getRow() != SIZE -1 -numStepsNorth || tur.getCol() != SIZE -1) {
        std::cout << "When direction is Direction::NORTH, move method of " <<
            numStepsNorth << " steps is wrong." << std::endl;
        break;
      }

      std::cout << "move method is correct" << std::endl;

      break;
    }
//expected output after running ./turtle G:
//move method is correct

    default:
      std::cout << "Unrecognized option: " << type << std::endl;
  }

  return 0;
}
```

Compile and link `Turtle.cpp` and `TurtleUnitTest.cpp` using the following command.

```
g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
```

Run each test case by passing the corresponding letter as a command-line argument. For example, to test the `reset` method, run

```
./turtle A
```

Test cases `A` through `G` correspond to the following methods:

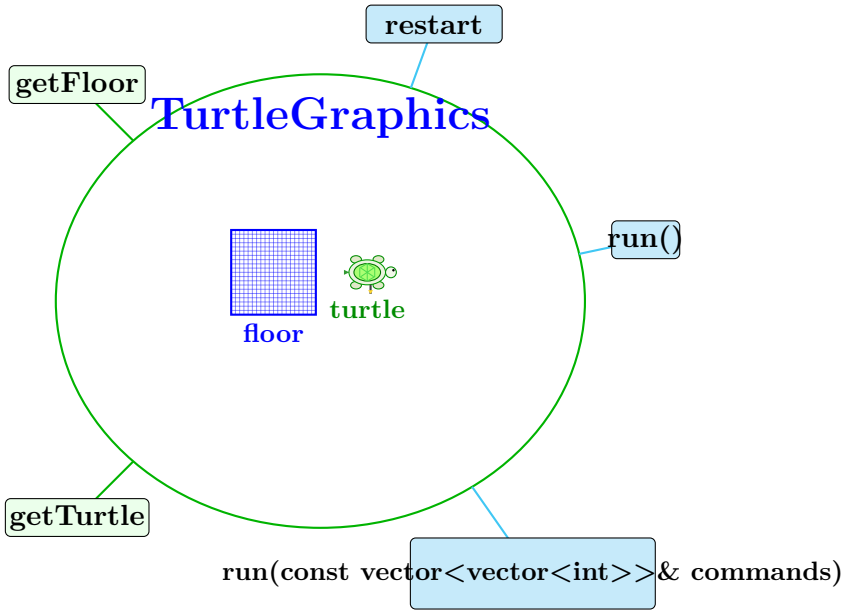
- `A` — `reset`
- `B` — `isPenUp`, `liftUpPen`, `putDownPen`
- `C` — `setRow`, `getRow`
- `D` — `setCol`, `getCol`
- `E` — `turnLeft`, `setHeading`, `getHeading`
- `F` — `turnRight`, `setHeading`, `getHeading`
- `G` — `move`

5.5 Submission for Task B

Submit `Turtle.cpp` to gradescope.

6 Task C: Implement TurtleGraphics Class

6.1 Illustration of TurtleGraphics Class



6.2 Download TurtleGraphics.hpp

Download link: TurtleGraphics.hpp

6.3 Implement TurtleGraphics.cpp

Read the requirements in `TurtleGraphics.hpp`. Implement all methods declared in `TurtleGraphics.hpp` inside `TurtleGraphics.cpp`.

6.4 Method run

There are two versions of `run` method in TurtleGraphics.cpp.

```
// Runs the turtle graphics program in interactive mode.
// Reads commands from the console (stdin) until the user
// enters 9 to stop. Calls restart() before processing commands.
// Valid commands:
//   1 - pen up
//   2 - pen down
//   3 - turn right
//   4 - turn left
//   5 - move (followed by number of steps)
```

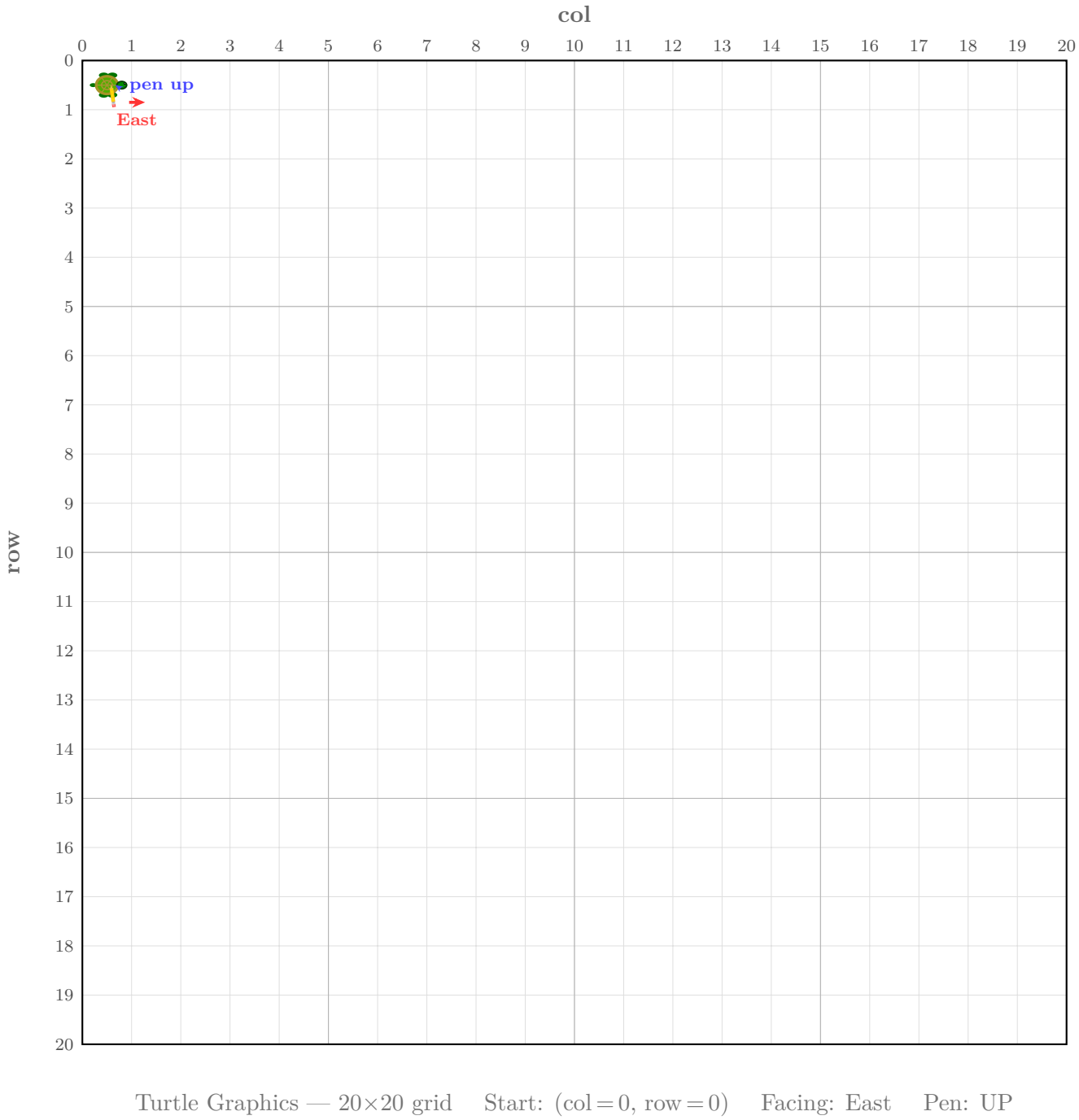
```
//   6 - print floor
//   9 - stop
run();

// Runs the turtle graphics program using a predefined list of commands.
// Each inner vector represents one command:
//   - commands with no parameter: {command}     e.g. {1}, {2}, {3}
//   - commands with one parameter: {command, steps} e.g. {5, 3}
// Calls restart() before processing commands.
// @param commands: a 2D vector of integer commands to execute
run(const std::vector<std::vector<int>>& commands);
```

6.4.1 Version 1: Reading Commands from the Console

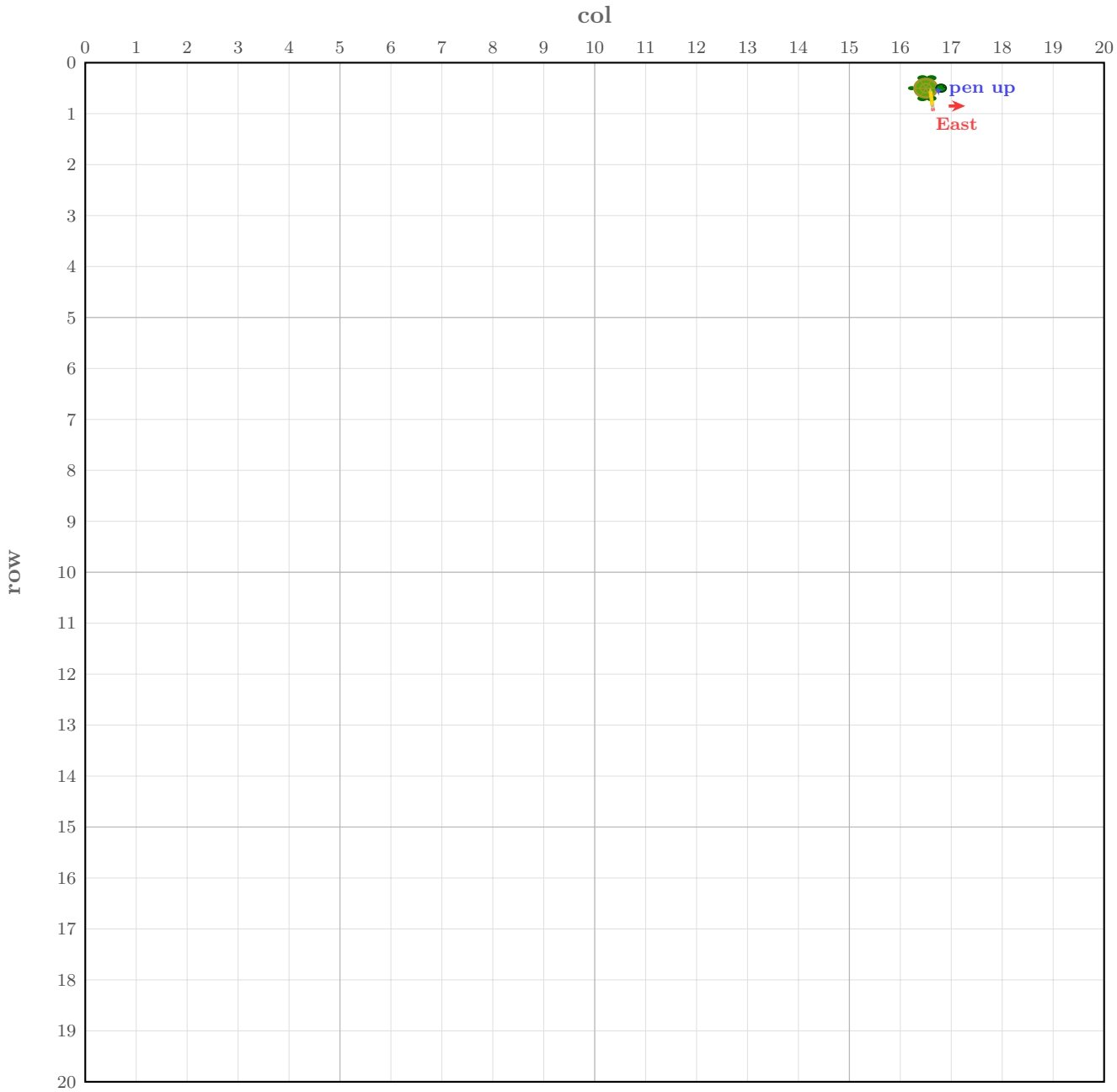
Suppose we do the following.

1. Let a turtle start from (0, 0) of a 20 x 20 grid, pen up, facing east.



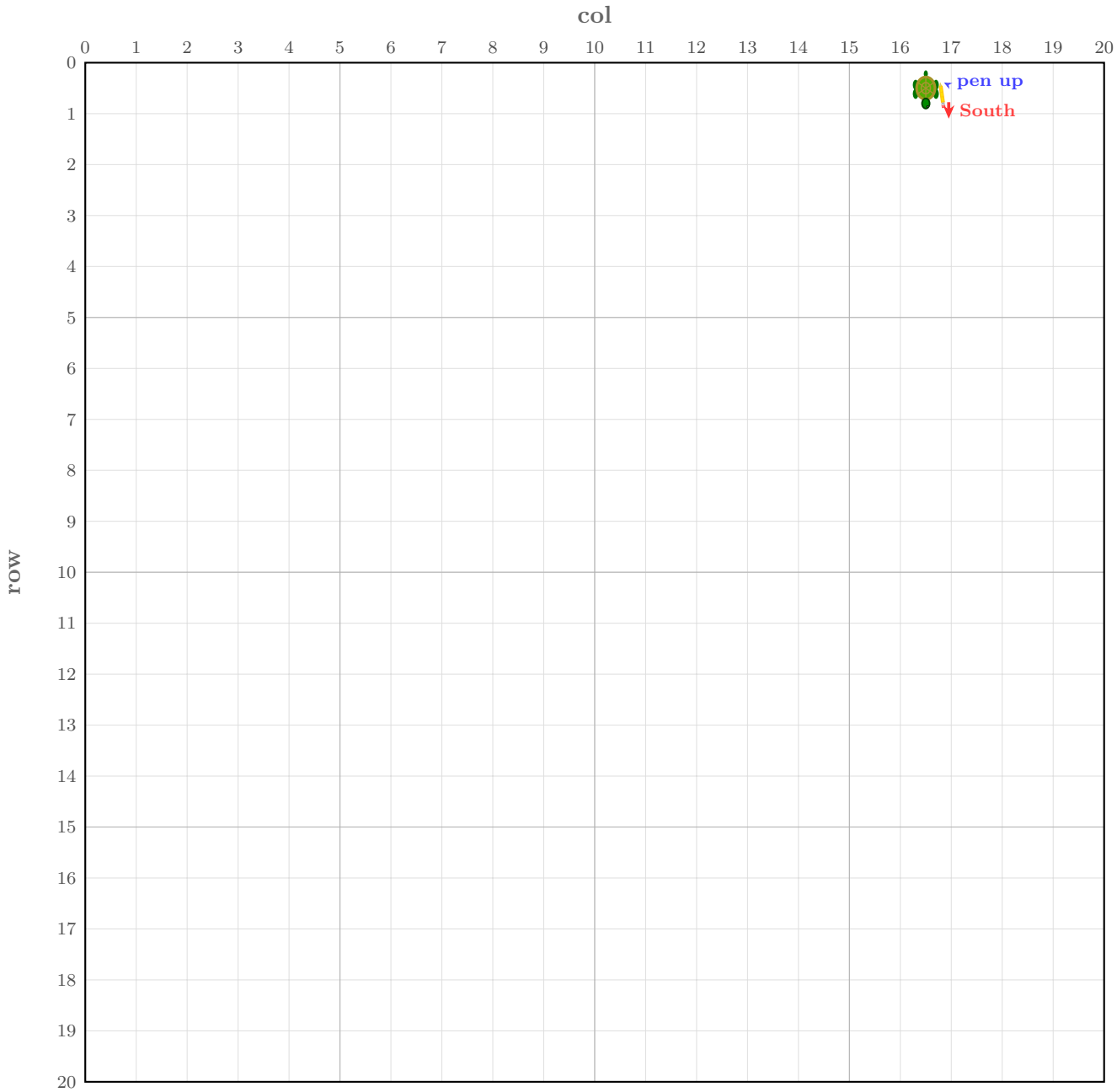
Turtle Graphics — 20×20 grid Start: (col = 0, row = 0) Facing: East Pen: UP

2. Enter command 5 (Move). This command requires a parameter for the number of steps; enter 16. The turtle then moves 16 steps east from its current position, ending at row and column indices (0, 16).



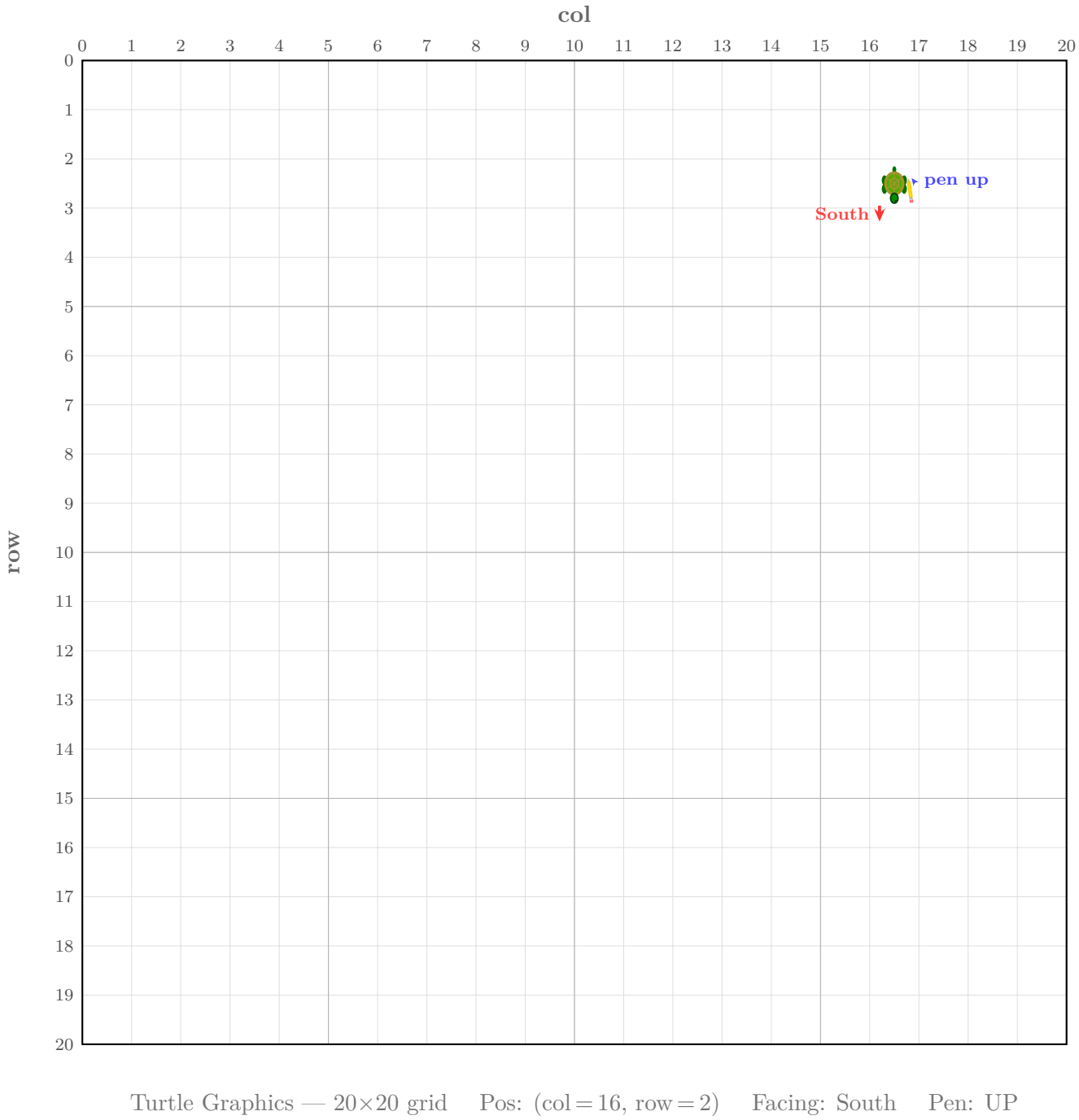
Turtle Graphics — 20×20 grid Start: (col = 16, row = 0) Facing: East Pen: UP

3. Enter command 3 (Turn Right). The turtle rotates 90 degrees clockwise, changing its heading from east to south.

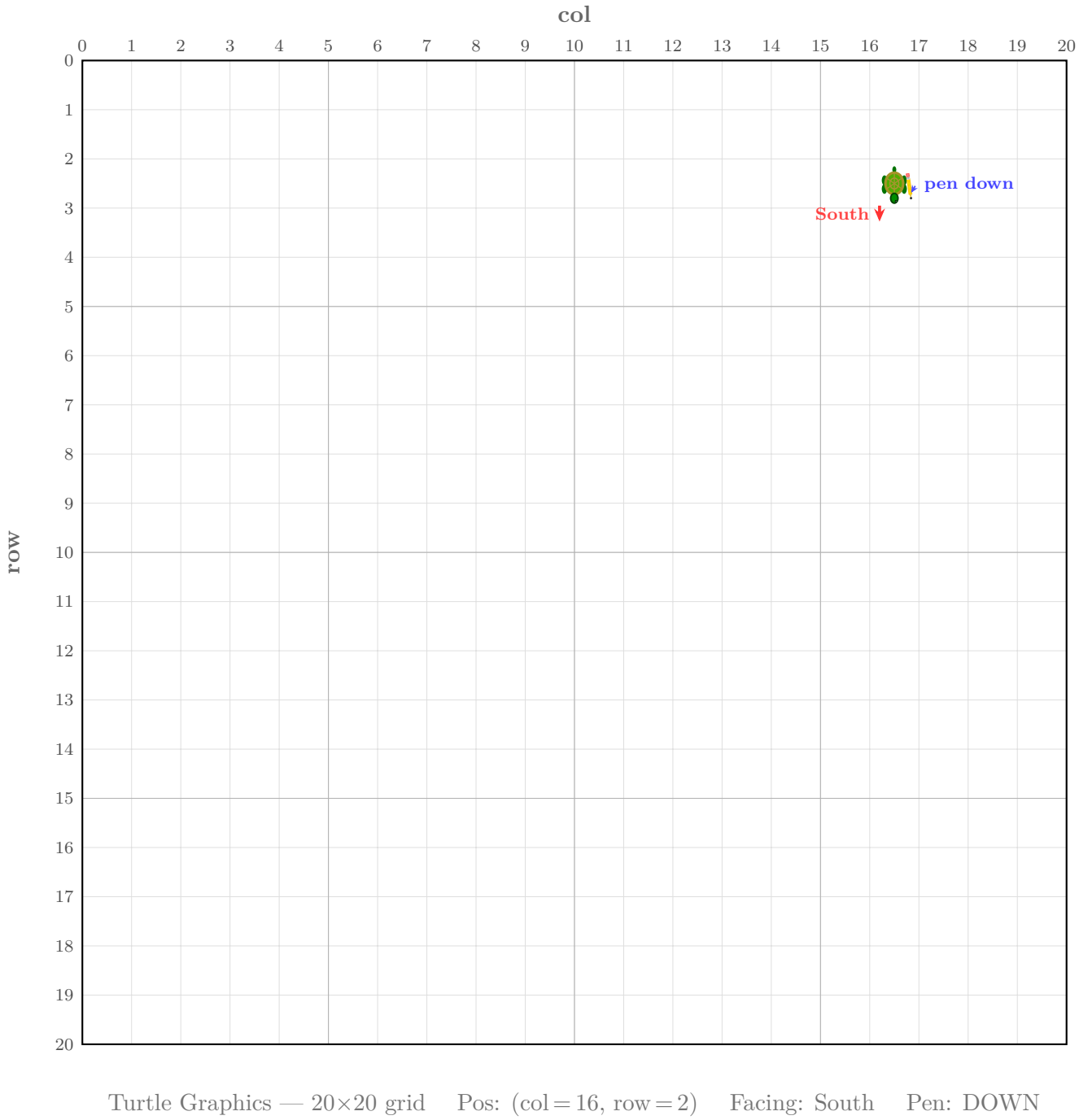


Turtle Graphics — 20×20 grid Pos: (col = 16, row = 0) Facing: South Pen: UP

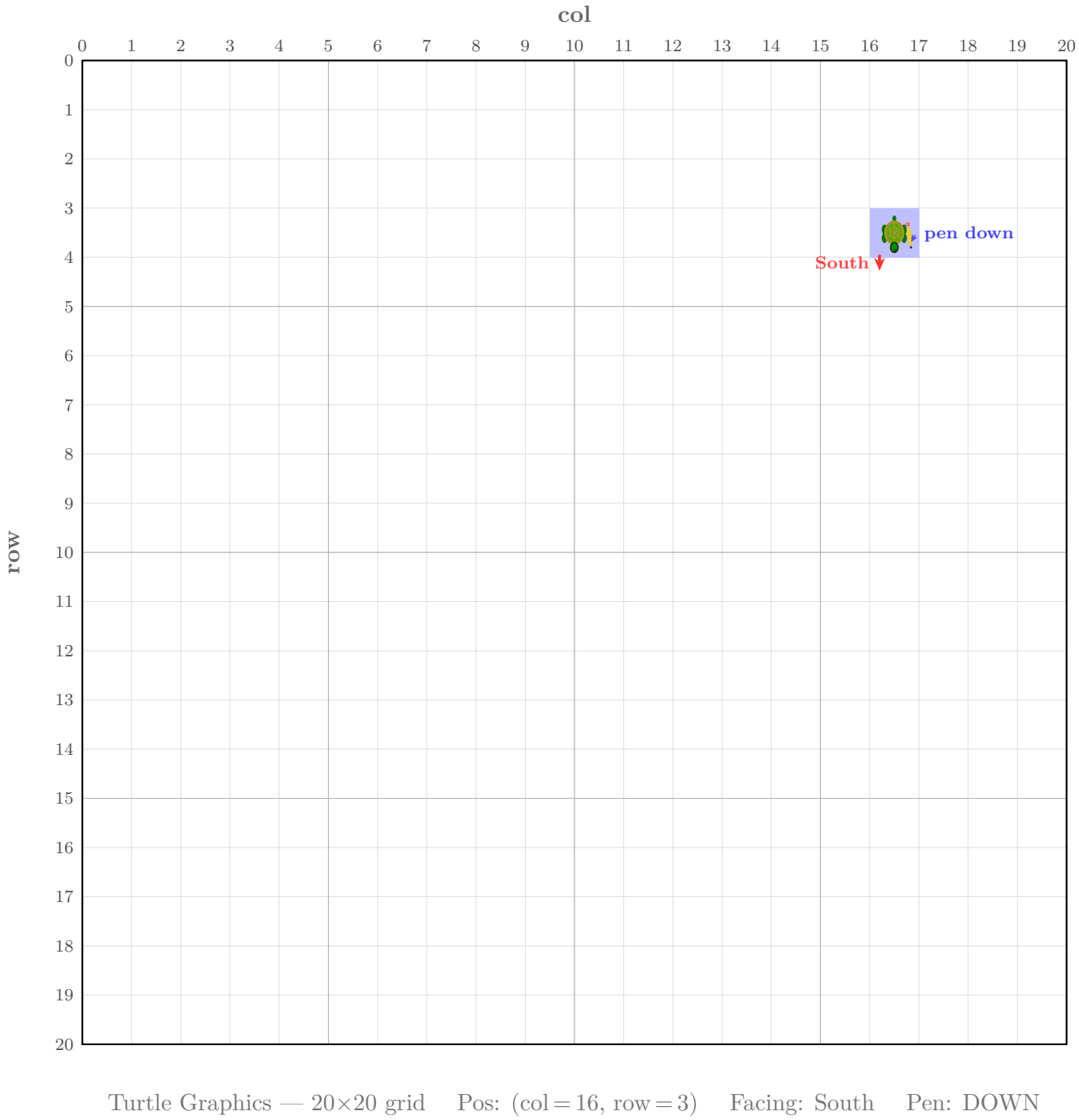
4. Enter command 5 (Move). This command requires an additional parameter for the number of steps; enter 2. The turtle then moves 2 steps in its current direction.



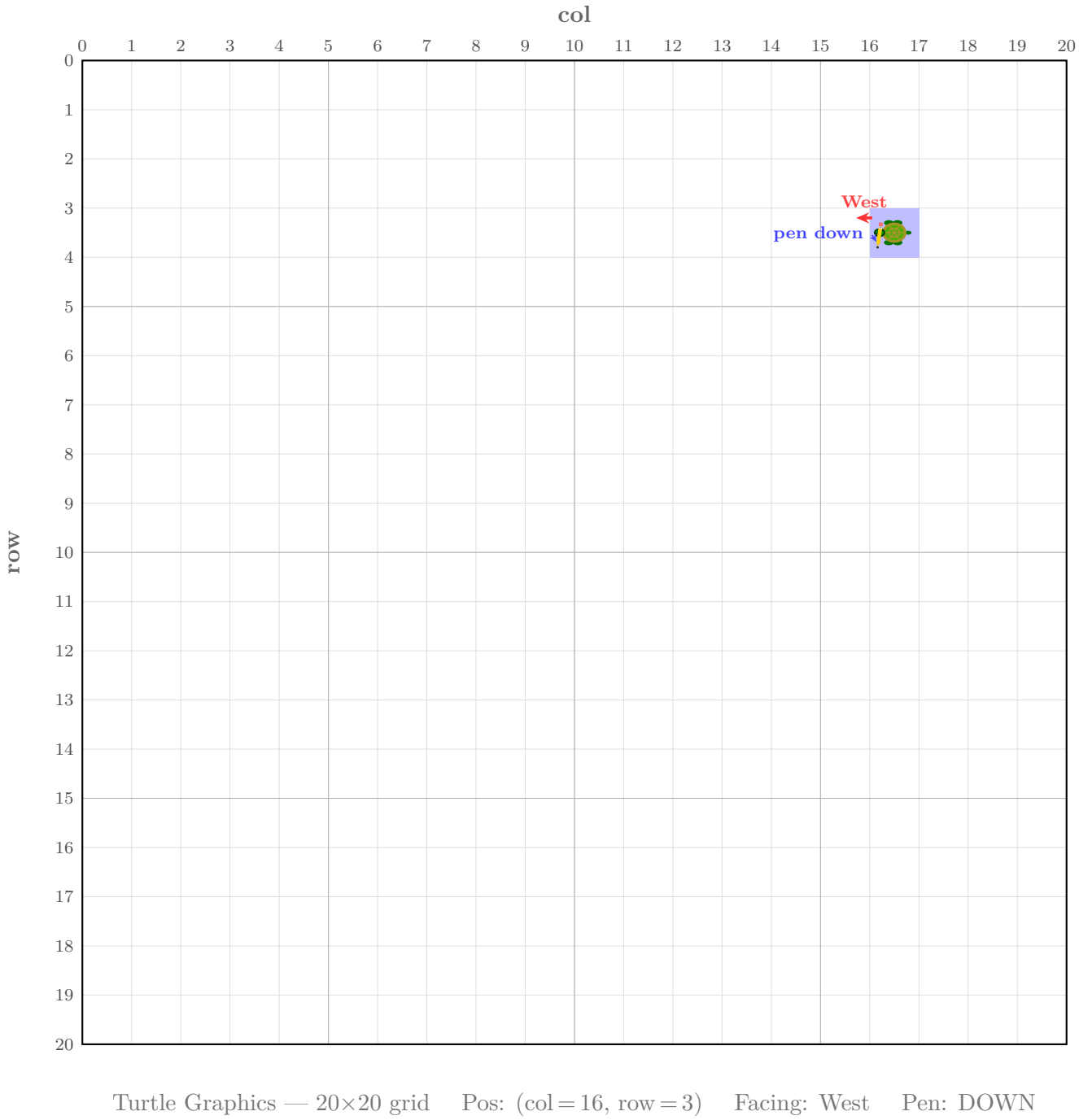
5. Enter command 2 (Pen Down). The turtle lowers the pen to the drawing surface.



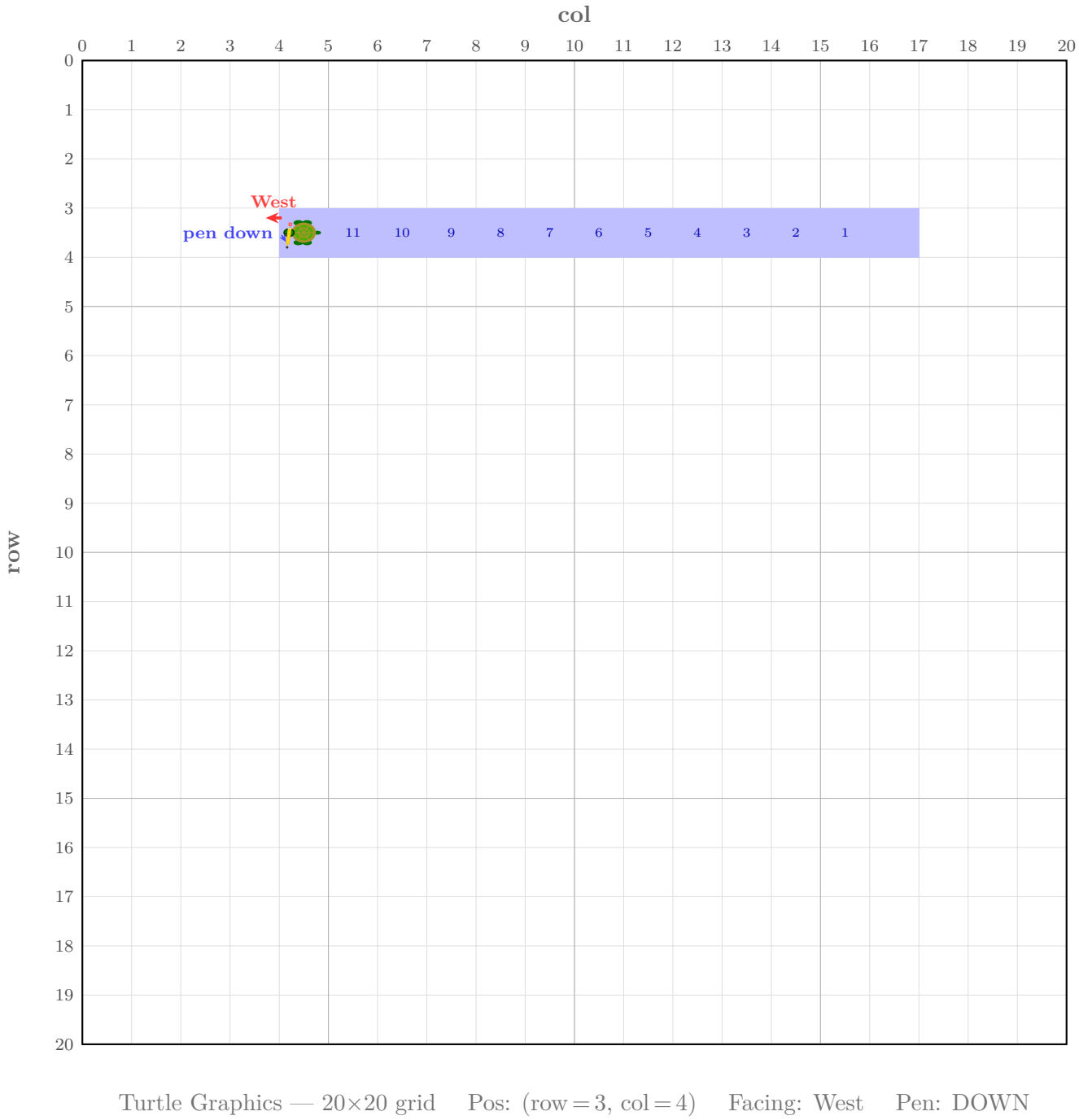
6. Execute command 5 (Move) with a parameter of 1. The turtle moves one step south. While the pen is down, it would typically leave a mark (such as an asterisk); however, to avoid being obscured by the turtle icon, the cell color changes to blue as a compromise.



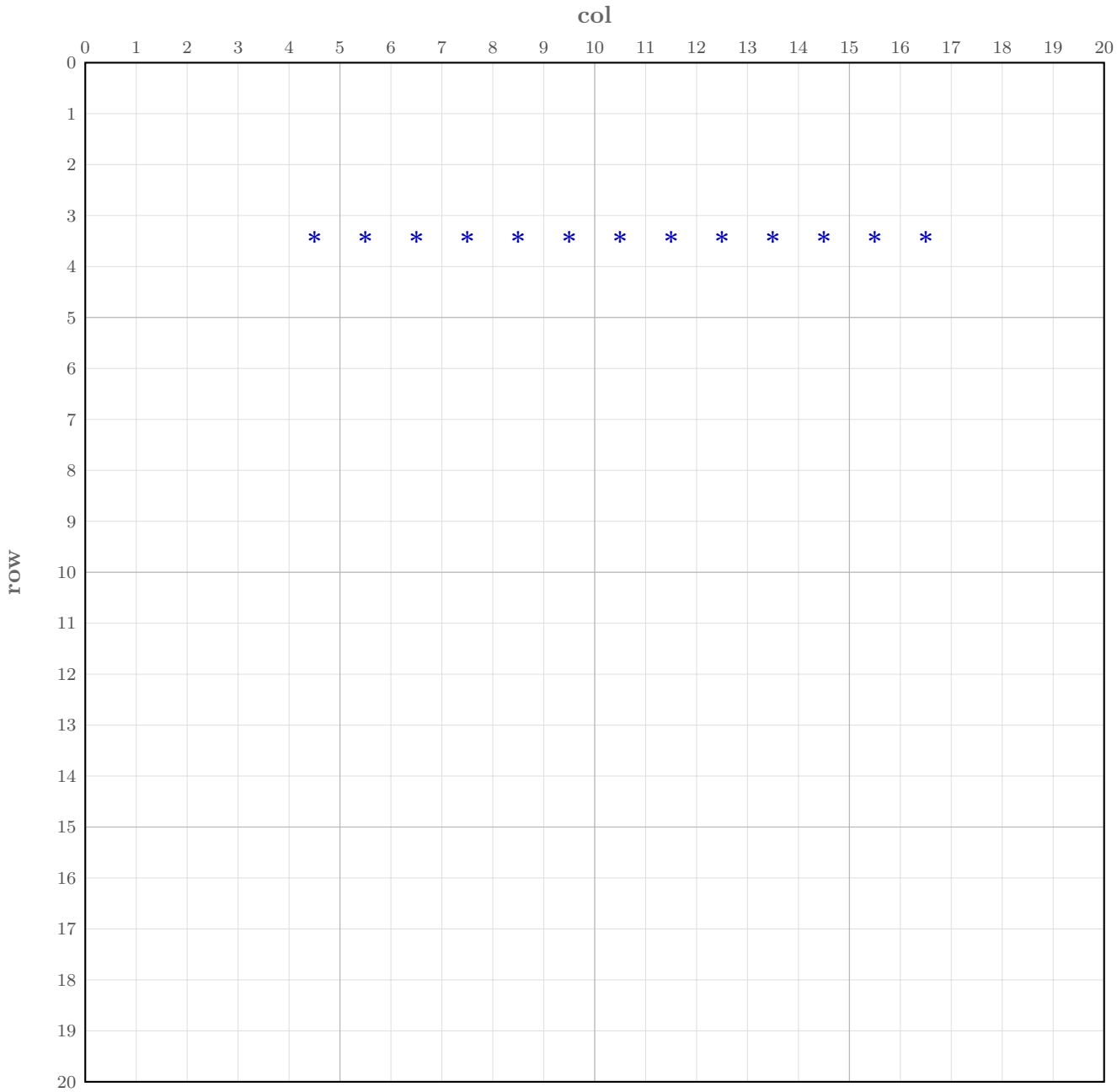
7. Execute command 3 (Turn Right). The turtle rotates clockwise, changing its heading from south to west.



8. Execute command 5 (Move) with a parameter of 12. The turtle moves 12 steps in the current direction, changing the color of each passing cell to blue.



9. Execute command 6 (Print). Display the current state of the floor grid.



Floor status after Command 6 (pen up) Painted: row = 3, col = 4 – 16

10. Execute command 9 to finish the game.

6.4.2 Version 2: Reading Commands from a Two-Dimensional Vector

The commands from Section 6.4.1 can be organized into a two-dimensional vector as follows. Each element of `commands` is a one-dimensional vector containing one or two integers enclosed in braces. Reason: command 5 (Move) needs an additional parameter to represent the number of steps to move.

```
std::vector<std::vector<int>> commands =
  { {5,16}, {3}, {5,2}, {2}, {5,1}, {3}, {5,12}, {6}, {9} };
```

6.4.3 Comparison of Run Method Versions 1 and 2

Both versions of the run() method must call the restart() method of the TurtleGraphics class. This clears the floor, places the turtle in the top-left corner, sets the pen to the "up" position, and ensures the turtle is facing east.

Logic	Version 1: Console	Version 2: 2D Vector
Execution	Read and process commands from the console sequentially until command 9 is entered.	Iterate through the 2D vector. For each inner vector, extract the first element as the command and process it.
Command Processing	If the command is 5, read one additional parameter from the console to move the turtle. For all other commands, execute the corresponding Turtle or Floor methods.	If the command is 5, use the second element of the inner vector as the movement parameter. For all other commands, execute the corresponding Turtle or Floor methods.

6.4.4 Hint for Method `processCommand`

The private method `processCommand` is called internally by both `run()` and `run(commands)` to reduce code redundancy. Notice that it has a parameter `inputFromConsole`:

- When called from `run()`, command 5 reads the number of steps from `std::cin`.
- When called from `run(commands)`, command 5 uses the `numSteps` parameter directly.

Think about what value of `inputFromConsole` corresponds to each case, and how `numSteps` should be set in each call.

6.5 Download input3.txt

These are the commands to draw digit 3 in a 20 x 20 grid.

Download link: **input3.txt**

6.6 Test TurtleGraphics.cpp

Download link: **TurtleGraphicsUnitTest.cpp**

Compile and link all source files using the following command.

```
g++ -o tg Floor.cpp Turtle.cpp TurtleGraphics.cpp TurtleGraphicsUnitTest.cpp
```

Run the resulting executable to verify your implementation.

Test default constructor of TurtleGraphics class using

```
./tg A
```

Test restart method using

```
./tg B
```

Test run commands stored in a two-dimensional vector to draw digit 5 using

```
./tg C
```

Test run commands with console input to draw digit 3 using

```
./tg D < input3.txt
```

Or you can test the commands by inputing one a time from the console using

```
./tg D
```

6.7 Submission for Task C

Submit `TurtleGraphics.cpp` to gradescope.

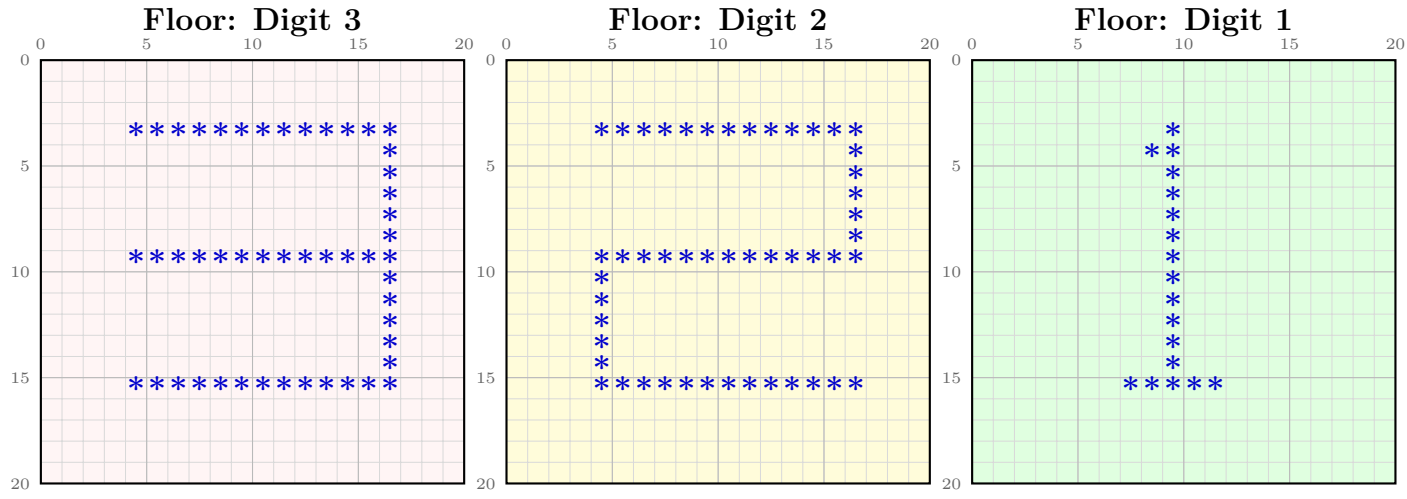
7 Task D: Draw Shapes Side by Side

Now we have defined Floor, Turtle, and TurtleGraphics classes, we would like to draw shapes side by side.

7.1 Drawing Digits of an Integer Side-by-Side

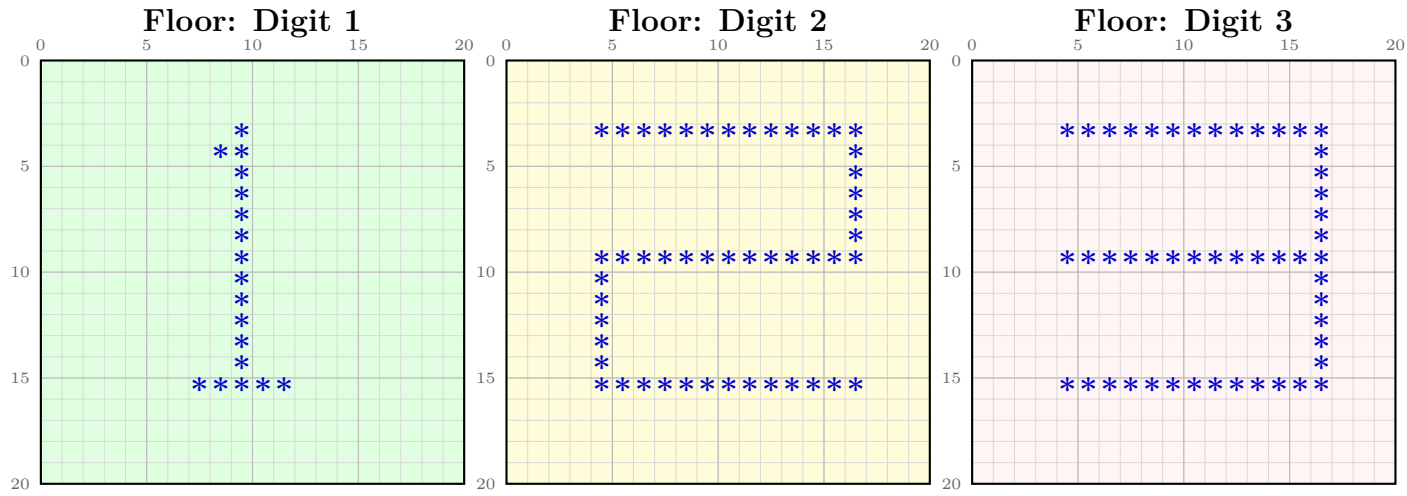
Given an integer, perform the following steps:

1. Extract the digits from right to left (from least significant to most significant).
2. For each digit, call the `run()` method of the `TurtleGraphics` class to draw the corresponding digit. After drawing, retrieve the floor object and add it to the end of a vector.
3. For example, if the input integer is `123`, the digit `3` is extracted and stored first, followed by `2`, and finally `1`. The resulting vector maintains this order, as illustrated below:



Stack order: digit 3 pushed first (leftmost floor) — digit 1 pushed last (rightmost floor)

However, we would like to show the original integer 123 as follows.



Floors left to right: digit 1, digit 2, digit 3

The rendering logic is outlined in the pseudocode below:

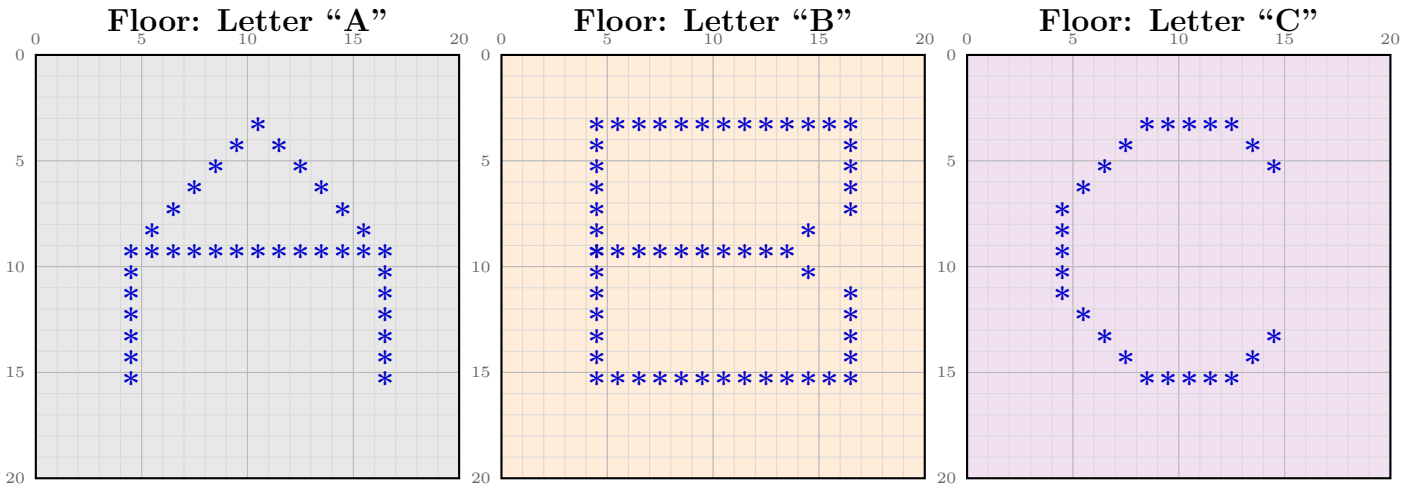
```
Outer loop: Iterate through each row of the floor matrix.

Middle loop: Iterate through the shapes in REVERSE order (from the last index to
    the first).
// Reason: Digits were stored least-significant-first.
// Reversing the order prints them from left to right (most significant digit first
    ).

Inner loop: Iterate through each column of the current shape's row.
```

For more details, see the `void display(int num)` method in the code snippet in Section 7.3.

7.2 Drawing Characters of a String Side-by-Side



Push order: A first (leftmost floor), B second, C last (rightmost floor)

When processing the characters of a string, the shapes are pushed into a vector from left to right, matching the final printing sequence. The rendering logic is outlined in the pseudocode below:

```
Outer loop: Iterate through each row of the floor matrix.

Middle loop: Iterate through the shapes in forward order (from the first index to the
    last).

Inner loop: Iterate through each column of the current shape's row.
```

For more details, see the `void display(const std::string& str)` method in the code snippet in Section 7.3.

7.3 Skeleton Code for `drawSideBySide.cpp`

Here is the skeleton of drawSideBySide.cpp, downloaded from **drawSideBySide.cpp**. Note that this file starts with a small letter, since it is not a file to define a class.

```cpp
//File: drawSideBySide.cpp
#include <iostream>
#include <string>
#include <vector>
#include <cctype> //isalpha, isdigit, isspace
#include <climits> //INT_MAX
#include "TurtleGraphics.hpp"

std::vector<std::vector<int>> CMDS_CAP_LETTERS[] = {
  //std::vector<std::vector<int>> commandsA =
  {
    // tip at (3,10): approach from (2,10) going south
    {1}, //pen up
    {3}, //turn right -> SOUTH (start facing EAST, right=SOUTH)
    {5,2}, //move south -> (2,0)... wait need to go east first
    {4}, //turn left -> EAST
    {5,10}, //move east -> (2,10)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (3,10)

    // left diagonal: (4,9),(5,8),(6,7),(7,6),(8,5)
    {1}, //pen up
    {3}, //turn right -> WEST
    {5,1}, //move west -> (3,9)
    {4}, //turn left -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (4,9)

    {1}, //pen up
```

```
    {3}, //turn right -> WEST
    {5,1}, //move west -> (4,8)
    {4}, //turn left -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (5,8)

    {1}, //pen up
    {3}, //turn right -> WEST
    {5,1}, //move west -> (5,7)
    {4}, //turn left -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (6,7)

    {1}, //pen up
    {3}, //turn right -> WEST
    {5,1}, //move west -> (6,6)
    {4}, //turn left -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (7,6)

    {1}, //pen up
    {3}, //turn right -> WEST
    {5,1}, //move west -> (7,5)
    {4}, //turn left -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (8,5)

    // right diagonal: (4,11),(5,12),(6,13),(7,14),(8,15)
    {1}, //pen up
    {4}, //turn left -> EAST (from SOUTH)
    {4}, //turn left -> NORTH
    {5,5}, //move north -> (3,5)
    {3}, //turn right -> EAST
    {5,6}, //move east -> (3,11)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (4,11)

    {1}, //pen up
    {4}, //turn left -> EAST
    {5,1}, //move east -> (4,12)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (5,12)

    {1}, //pen up
```

```
    {4}, //turn left -> EAST
    {5,1}, //move east -> (5,13)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (6,13)

    {1}, //pen up
    {4}, //turn left -> EAST
    {5,1}, //move east -> (6,14)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (7,14)

    {1}, //pen up
    {4}, //turn left -> EAST
    {5,1}, //move east -> (7,15)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,1}, //draw * at (8,15)

    // crossbar row 9, cols 4-16
    {1}, //pen up
    {5,1}, //move south -> (8,15) ... need to navigate to (9,3)
    {3}, //turn right -> WEST
    {5,12}, //move west -> (8,3)
    {4}, //turn left -> SOUTH
    {4}, //turn left -> EAST
    {2}, //pen down
    {5,13}, //draw crossbar cols 4-16

    // left leg col 4, rows 10-15
    {1}, //pen up
    {4}, //turn left -> NORTH (from EAST)
    {4}, //turn left -> WEST
    {5,12}, //move west -> (9,4)
    {3}, //turn right -> NORTH
    {3}, //turn right -> EAST
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,6}, //draw left leg rows 10-15

    // right leg col 16, rows 10-15
    {1}, //pen up
    {4}, //turn left -> EAST
    {4}, //turn left -> NORTH
    {5,6}, //move north -> (9,4)... need (9,16)
```

```
    {3}, //turn right -> EAST
    {5,12}, //move east -> (9,16)
    {3}, //turn right -> SOUTH
    {2}, //pen down
    {5,6}, //draw right leg rows 10-15

    {1}, //pen up
    {6}, //print
  },

  //TurtleGraphics tgA;
  //tgA.run(commandsA);

  //std::vector<std::vector<int>> commandsB =
  { // pen up, move to (2,4), draw left vertical col4 rows 3-15
    {1}, //pen up
    {5,4}, //move east 4 steps
    {3}, //turn right, from east to south
    {5,2}, //move south 2 steps -> (2,4)
    {2}, //pen down
    {5,13}, //draw left vertical from (3,4) to (15,4)

    // bottom horizontal row15 cols 5-16
    {4}, //turn left, from south to east
    {5,12}, //draw bottom horizontal cols 5-16

    // bottom right col16 rows 14-11
    {1}, //pen up
    {4}, //turn left, from east to north
    {2}, //pen down
    {5,4}, //draw col16 rows 14 down to 11

    // bottom curve at (10,14)
    {1}, //pen up
    {5,1}, //move north 1 -> (10,16)
    {4}, //turn left, from north to west
    {5,2}, //move west 2 -> (10,14)
    {3}, //turn right, from west to north
    {5,1}, //move north 1 -> (9,14)
    {3}, //turn right, from north to east
    {3}, //turn right, from east to south
    {2}, //pen down
    {5,1}, //draw bottom curve at (10,14)

    // middle horizontal row9 cols 4-13
    {1}, //pen up
```

```
    {3}, //turn right, from south to west
    {3}, //turn right, from west to north
    {5,1}, //move north 1 -> (9,14)
    {3}, //turn right, from north to east
    {3}, //turn right, from east to south
    {3}, //turn right, from south to west
    {5,11}, //move west 11 -> (9,3)
    {3}, //turn right, from west to north
    {3}, //turn right, from north to east
    {2}, //pen down
    {5,10}, //draw middle horizontal cols 4-13

    // top curve at (8,14)
    {1}, //pen up
    {3}, //turn right, from east to south
    {3}, //turn right, from south to west
    {3}, //turn right, from west to north
    {5,1}, //move north 1 -> (8,13)
    {3}, //turn right, from north to east
    {2}, //pen down
    {5,1}, //draw top curve at (8,14)

    // top right col16 rows 3-7
    {1}, //pen up
    {5,2}, //move east 2 -> (8,16)
    {4}, //turn left, from east to north
    {2}, //pen down
    {5,5}, //draw col16 rows 7 down to 3

    // top horizontal row3 cols 5-15
    {1}, //pen up
    {3}, //turn right, from north to east
    {3}, //turn right, from east to south
    {3}, //turn right, from south to west
    {2}, //pen down
    {5,11}, //draw top horizontal cols 15 down to 5

    {1}, //pen up
    {6}, //print
  },
  //tgA.run(commandsB);

  //std::vector<std::vector<int>> commandsC =
  { {1},{3},{5,3},{4},{5,7},{2},{5,5},      // top horiz row3 cols 8-12
    {1},{5,1},{3},{2},{5,1},                // top-right (4,13)
    {1},{4},{5,1},{3},{2},{5,1},            // top-right (5,14)
```

```
    {1},{4},{4},{5,2},{4},{5,7},{4},{2},{5,1}, // top-left (4,7)
    {1},{3},{5,1},{4},{2},{5,1},              // top-left (5,6)
    {1},{3},{5,1},{4},{2},{5,1},              // top-left (6,5)
    {1},{3},{5,1},{4},{2},{5,5},              // left vert col4 rows 7-11
    {1},{4},{5,1},{3},{2},{5,1},              // bot-left (12,5)
    {1},{4},{5,1},{3},{2},{5,1},              // bot-left (13,6)
    {1},{4},{5,1},{3},{2},{5,1},              // bot-left (14,7)
    {1},{5,1},{4},{2},{5,5},                  // bot horiz row15 cols 8-12
    {1},{4},{5,4},{3},{5,3},{3},{1},{5,1},   // bot-right (12,15), change from {2},{5,1}
        to {1},{5,1}, do not display the top * in the bottom right diagonal
    {1},{3},{5,1},{4},{2},{5,1},              // bot-right (13,14)
    {1},{3},{5,1},{4},{2},{5,1},              // bot-right (14,13)
    {1},{6}
  },
  //tgA.run(commandsC);

  //std::vector<std::vector<int>> commandsD =
  { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13},  // left vert col4 rows 3-15
    {1},{4},{4},{5,12},{3},{2},{5,6},         // top horiz row3 cols 5-10
    {1},{5,2},{3},{2},{5,1},                  // top curve (4,12)
    {1},{4},{5,2},{3},{2},{5,1},              // top curve (5,14)
    {1},{4},{5,2},{3},{2},{5,6},              // right vert col16 rows 6-11
    {1},{2},{5,1},                            // bot curve (12,16)
    {1},{3},{5,2},{4},{2},{5,1},              // bot curve (13,14)
    {1},{3},{5,2},{4},{2},{5,1},              // bot curve (14,12)
    {1},{5,1},{3},{5,8},{4},{4},{2},{5,6},     // bot horiz row15 cols 5-10
    {1},{6}
  },
  //tgA.run(commandsD);

  //std::vector<std::vector<int>> commandsE =
  { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13}, // left vert col4 rows 3-15
    {1},{4},{4},{5,12},{4},{5,1},{4},{4},{2},{5,13}, // top horiz row3 cols 4-16
    {1},{3},{5,6},{3},{5,13},{4},{4},{2},{5,11}, // middle horiz row9 cols 4-14
    {1},{3},{5,6},{3},{5,11},{4},{4},{2},{5,13}, // bot horiz row15 cols 4-16
    {1},{6}
  },
  //tgA.run(commandsE);

  //std::vector<std::vector<int>> commandsF =
  { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13}, // left vert col4 rows 3-15
    {1},{4},{4},{5,12},{4},{5,1},{4},{4},{2},{5,13}, // top horiz row3 cols 4-16
    {1},{3},{5,6},{3},{5,13},{4},{4},{2},{5,11}, // middle horiz row9 cols 4-14
    {1},{6}
  },
  //tgA.run(commandsF);
```

```
260
261 //std::vector<std::vector<int>> commandsG =
262 { // left vertical col4 rows 3-15
263   {1}, //pen up
264   {5,4}, //move east 4 steps
265   {3}, //turn right, from east to south
266   {5,2}, //move south 2 -> (2,4)
267   {2}, //pen down
268   {5,13}, //draw left vertical rows 3-15
269
270   // bottom horizontal row15 cols 5-16
271   {4}, //turn left, from south to east
272   {2}, //pen down
273   {5,12}, //draw bottom horizontal cols 5-16
274
275   // right vertical col16 rows 14-9
276   {1}, //pen up
277   {4}, //turn left, from east to north
278   {2}, //pen down
279   {5,6}, //draw col16 rows 14 down to 9
280
281   // middle arm row9 cols 15-10
282   {4}, //turn left, from north to west
283   {5,6}, //draw middle arm cols 15 down to 10
284
285   // top-right curve col16 rows 5-3
286   {3}, //turn right, from west to north
287   {1}, //pen up
288   {5,3}, //move north 3 -> (6,10)
289   {3}, //turn right, from north to east
290   {5,6}, //move east 6 -> (6,16)
291   {3}, //turn right, from east to south
292   {3}, //turn right, from south to west
293   {3}, //turn right, from west to north
294   {2}, //pen down
295   {5,3}, //draw col16 rows 5,4,3
296
297   // top horizontal row3 cols 15-5
298   {1}, //pen up
299   {3}, //turn right, from north to east
300   {3}, //turn right, from east to south
301   {3}, //turn right, from south to west
302   {2}, //pen down
303   {5,11}, //draw top horizontal cols 15 down to 5
304
305   {1}, //pen up
```

```
    {6}, //print
  },
  //tgA.run(commandsG);

  //std::vector<std::vector<int>> commandsH =
  { {1},{5,4},{3},{5,2},{2},{5,13},          // left vertical col4 rows 3-15
    {1},{4},{5,12},{4},{5,13},{3},{3},{2},{5,13}, // right vertical col16 rows 3-15
    {1},{3},{3},{5,6},{3},{3},{3},{2},{5,12}, // middle horiz row9 cols 5-16
    {1},{6},
  },
  //tgA.run(commandsH);

  //std::vector<std::vector<int>> commandsI =
  { {1},{3},{5,3},{4},{5,4},{2},{5,11},      // top horiz row3 cols 5-15
    {1},{4},{5,1},{4},{5,5},{4},{2},{5,13},   // center vert col10 rows 3-15
    {1},{3},{5,6},{4},{4},{2},{5,11},         // bot horiz row15 cols 5-15
    {1},{6}
  },
  //tgA.run(commandsI);

  //std::vector<std::vector<int>> commandsJ =
  { // top horiz row3 cols 5-15 (11 cells)
    {1},{3},{5,3},{4},{5,4},{2},{5,11},
    // vert col10 rows 3-12
    {1},{4},{5,1},{4},{5,5},{4},{2},{5,10},
    // curve (13,9)
    {1},{3},{5,1},{4},{2},{5,1},
    // curve (14,8)
    {1},{3},{5,1},{4},{2},{5,1},
    // left hook col4 rows 12-15
    {1},{4},{4},{5,3},{4},{5,4},{4},{2},{5,4},
    // bot horiz row15 cols 4-7
    {1},{3},{5,1},{4},{4},{2},{5,4},
    {1},{6}
  },
  //tgA.run(commandsJ);

  //std::vector<std::vector<int>> commandsK =
  { // left vertical col4 rows 3-15
    {1},{3},{5,2},{4},{5,4},{3},{2},{5,13},
    // upper diagonal: (8,6),(7,8),(6,10),(5,12),(4,14) -- step 2 per row
    {1},{4},{4},{5,8},{3},{5,2},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
```

```
    // lower diagonal: (10,6),(11,8),(12,10),(13,12),(14,14) -- step 2 per row
    {1},{5,5},{3},{5,8},{4},{2},{5,1},
    {1},{4},{5,2},{3},{2},{5,1},
    {1},{4},{5,2},{3},{2},{5,1},
    {1},{4},{5,2},{3},{2},{5,1},
    {1},{4},{5,2},{3},{2},{5,1},
    {1},{6}
  },
  //tgA.run(commandsK);

  //std::vector<std::vector<int>> commandsL =
  //letter L
  { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {3}, {5, 1}, {4}, {4}, {2},
     {5, 13}, {1}, {6}
  },
  //tgA.run(commandsL);

  //std::vector<std::vector<int>> commandsM =
  { {{1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
      12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5, 11}, {4}, {2}, {5, 1},
     {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4},
     {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1},
     {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 6}, {3}, {5, 5}, {3}, {2}, {5, 1}, {1}, {3},
     {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1},
     {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2},
     {5, 1}, {1}, {6}}
  },
  //tgA.run(commandsM);

  //std::vector<std::vector<int>> commandsN =
  { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
     12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5, 11}, {4}, {2}, {5, 1},
     {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4},
     {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1},
     {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2},
     {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1},
     {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {6}
  },
  //tgA.run(commandsN);


  //std::vector<std::vector<int>> commandsO =
  { // top horiz row3 cols 8-12
    {1},{3},{5,3},{4},{5,7},{2},{5,5},
    // top-right corner: (4,13),(5,14),(6,15)
    {1},{5,1},{3},{2},{5,1},
```

```
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    // right vert col16 rows 7-11
    {1},{4},{5,1},{3},{2},{5,5},
    // bot-right corner: (12,15),(13,14),(14,13)
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
    // bot horiz row15 cols 8-12
    {1},{5,1},{3},{5,6},{4},{4},{2},{5,5},
    // bot-left corner: (14,7),(13,6),(12,5)
    {1},{4},{5,2},{4},{5,5},{4},{2},{5,1},
    {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
    {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
    // left vert col4 rows 11-7
    {1},{3},{5,1},{3},{2},{5,5},
    // top-left corner: (6,5),(5,6),(4,7)
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{5,2},{3},{5,1},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1},
    {1},{6}
  },
  //tgA.run(commands0);

  //std::vector<std::vector<int>> commandsP =
  { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5,
      1}, {4}, {4}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {4}, {2}, {5, 5}, {1}, {5, 1},
      {3}, {5, 1}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {5, 11}, {4}, {4}, {2}, {5, 10},
      {1}, {6}
  },
  //tgA.run(commandsP);

  //std::vector<std::vector<int>> commandsQ =
  { // 3-step round 0 (same as 0)
    {1},{3},{5,3},{4},{5,7},{2},{5,5},
    {1},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,5},
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{5,1},{3},{5,6},{4},{4},{2},{5,5},
    {1},{4},{5,2},{4},{5,5},{4},{2},{5,1},
    {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
    {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
```

```
    {1},{3},{5,1},{3},{2},{5,5},
    {1},{3},{5,1},{4},{2},{5,1},
    {1},{5,2},{3},{5,1},{3},{2},{5,1},
    {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1},
    // diagonal tail (9,10)..(15,16)
    {1},{5,4},{4},{5,3},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{4},{5,1},{3},{2},{5,1},
    {1},{6}
  },
  //tgA.run(commandsQ);

  //std::vector<std::vector<int>> commandsR =
  {
    {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5,
       1}, {4}, {4}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {4}, {2}, {5, 5}, {1}, {5,
       1}, {3}, {5, 1}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {5, 11}, {4}, {4}, {2}, {5,
        10}, {1}, {4}, {4}, {5, 2}, {4}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5,
       1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1},
        {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {6}
  },
  //tgA.run(commandsR);

  //std::vector<std::vector<int>> commandsS =
  { // top horiz row3 cols 6-14
    {1},{3},{5,3},{4},{5,5},{2},{5,9},
    // top-left 2-step: (4,5),(5,4)
    {1},{4},{4},{5,9},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
    // left vert col4 row 6 only
    {1},{2},{5,1},
    // 5 waist segments (each 2 cells, adjacent)
    {1},{5,1},{4},{2},{5,2},      // row7: cols 5-6
    {1},{3},{5,1},{4},{2},{5,2},  // row8: cols 7-8
    {1},{3},{5,1},{4},{2},{5,2},  // row9: cols 9-10
    {1},{3},{5,1},{4},{2},{5,2},  // row10: cols 11-12
    {1},{3},{5,1},{4},{2},{5,3},  // row11: cols 13-15
    // right vert col16 rows 12-13 only
    {1},{5,1},{3},{2},{5,2},
    // bot-right 2-step: (13,16),(14,15)
    {1},{4},{4},{5,1},{4},{4},{2},{5,1},
    {1},{3},{5,1},{4},{2},{5,1},
```

```
    // bot horiz row15 cols 6-14
    {1},{5,1},{3},{5,10},{4},{4},{2},{5,9},
    {1},{6}
  },
  //tgA.run(commandsS);

  //std::vector<std::vector<int>> commandsT =
  { {1}, {3}, {5, 3}, {4}, {5, 3}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {5, 6}, {4},
     {2}, {5, 13}, {1}, {6}
  },
  //tgA.run(commandsT);

  //std::vector<std::vector<int>> commandsU =
  { {1},{3},{5,2},{4},{5,4},{3},{2},{5,10}, // left vert col4 rows 3-12
    {1},{4},{4},{5,10},{3},{5,12},{3},{2},{5,10}, // right vert col16 rows 3-12
    {1},{3},{5,11},{4},{2},{5,1},           // bot-left (13,5)
    {1},{4},{5,1},{3},{2},{5,1},            // bot-left (14,6)
    {1},{5,1},{4},{2},{5,6},                // bot horiz row15 cols 7-12
    {1},{4},{5,2},{3},{5,2},{3},{2},{5,1},  // bot-right (14,14)
    {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1}, // bot-right (13,15)
    {1},{6}
  },
  //tgA.run(commandsU);

  //std::vector<std::vector<int>> commandsV =
  { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5,
     12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 12}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1},
      {3}, {5, 12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 11}, {4}, {2}, {5, 1}, {1}, {4},
     {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 10}, {4}, {2}, {5, 1},
     {1}, {4}, {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9}, {4}, {2},
      {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 8},
     {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3},
     {5, 7}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1},
     {1}, {3}, {5, 6}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2},
     {5, 1}, {1}, {3}, {5, 5}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 4},
     {3}, {2}, {5, 1}, {1}, {3}, {5, 4}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3},
     {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5,
     1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 2}, {4}, {2}, {5, 1}, {1}, {4},
     {4}, {5, 1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1},
     {1}, {6}
  },
  //tgA.run(commandsV);

  //std::vector<std::vector<int>> commandsW =
  { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
     12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 7}, {4}, {5, 6}, {4}, {2}, {5, 1}, {1},
```

```
     {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 2}, {3}, {2}, {5,
    1}, {1}, {3}, {5, 3}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 4}, {3},
    {2}, {5, 1}, {1}, {3}, {5, 5}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5,
    6}, {3}, {2}, {5, 1}, {1}, {3}, {5, 7}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1},
    {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9}, {4}, {2}, {5, 1}, {1}, {4}, {4},
    {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {6}
},
//tgA.run(commandsW);

//std::vector<std::vector<int>> commandsX =
{ {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5,
    12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 11}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1},
     {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9}, {4}, {2}, {5, 1}, {1}, {4},
    {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 7}, {4}, {2}, {5, 1},
    {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1}, {1}, {3}, {5, 5}, {4}, {2},
    {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3},
    {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3},
    {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5,
    1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3}, {4}, {2}, {5, 1}, {1}, {4},
    {4}, {5, 1}, {3}, {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 5}, {4}, {2}, {5, 1},
    {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1}, {1}, {3}, {5, 7}, {4}, {2},
    {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9},
    {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3},
     {5, 11}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 12}, {3}, {2}, {5, 1},
     {1}, {6}
},
//tgA.run(commandsX);

//std::vector<std::vector<int>> commandsY =
{ {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5,
    12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 11}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1},
     {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9}, {4}, {2}, {5, 1}, {1}, {4},
    {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 7}, {4}, {2}, {5, 1},
    {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1}, {1}, {3}, {5, 5}, {4}, {2},
    {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3},
    {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3},
    {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5,
    1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3}, {4}, {2}, {5, 1}, {1}, {4},
    {4}, {5, 1}, {3}, {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 5}, {4}, {2}, {5, 1},
    {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1}, {1}, {3}, {5, 7}, {4}, {2},
    {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9},
    {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3},
    {5, 1}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {4}, {4}, {2}, {5, 7}, {1}, {6}
},

//std::vector<std::vector<int>> commandsZ =
{ {1}, {3}, {5, 3}, {4}, {5, 3}, {2}, {5, 13}, {1}, {3}, {5, 12}, {3}, {5, 13}, {4},
    {4}, {2}, {5, 13}, {1}, {4}, {5, 12}, {4}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5,
     1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4},
    {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5,
    1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1},
    {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5,
```

```
    1}, {4}, {2}, {5, 1}, {1}, {6}
  },
  //tgA.run(commandsZ);
};

std::vector<std::vector<int>> CMDS_DIGITS[] = {
  //commands0: comments to draw 0
  { {1},  // pen up, facing east by default
    {5,4}, // move east 4 steps from (0,1) to (0,4)
    {3},  // turn right, from east to south
    {5,3}, // move 3 steps, from (1,4) to (3,4)
    {4},  // turn left, from south to east again

    {2},  // pen down

    // draw top horizontal line
    {5,12},// move 12 steps, from

    // draw right vertical line
    {3},{5,12},

    // draw bottom horizontal line
    {3},{5,12},

    // draw left vertical line
    {3},{5,12},

    // print
    {6}
  },

  //commands1: comments to draw 1
  { {1},{5,3},{3},{5,3},{4},
    {5,5},
    {2},{5,1},{3},{5,1},{3},{5,1},
    {1},{3},{5,1},{3},{5,1},{3},{5,1},
    {2},{5,11},
    {4},{5,2},{1},{4},{5,1},{4},{5,2},
    {2},{4},{5,1},{3},{5,2},
    {6}
  },

  //commands2
  {  {1},{5,3},{3},{5,3},{4},
     {2},{5,13},{3},{5,6},{3},{5,12},{4},{5,6},{4},{5,12},
     {6}
```

```
  },

  //commands3
  {  {1},{5,3},{3},{5,3},{4},
     {2},{5,13},{3},{5,6},
    {3},{5,12},{1},
    {4},{5,5},{2},{5,1},{4},
    {5,12},{4},{5,6},{6}
  },

  //commands4
  { {1},  // pen up, facing east by default
    {5,4}, // move east 4 steps from (0,1) to (0,4), where 4-1+1 = 4
    {3},  // turn right, from east to south
    {5,2}, // move 2 steps, from (1,4) to (2,3), where 2-1+1 = 2

    {2},  // pen down

    // draw left line
    {5,7},// move 12 steps, from (3,4) to (9,4), where 9-3+1 = 7

    {4}, // turn left, from south to east
    {5,12}, // move from (9,5) to (9,16), a total of of 16-5+1 = 12 steps

    // move to one step above the top of vertical line, ie, (2,16), facing south
    {1}, // pen up
    {4}, // turn left, from east to north
    {5,7}, // move from (8,16) to (2,16), a total of 8-2+1 = 7 steps
    {3}, // turn right, from north to east
    {3}, // turn right, from east to south

    {2}, // pen down
    {5,13}, // move from (3,16) to (15,16), a total of 15-3+1 = 13 steps

    // print
    {6}
  },

  //commands5
  { // facing east in the beginning
    {1}, // pen up
    {5,16}, // move from (0,1) to (0,16)
    {3}, // turn right from east to south
    {5,2}, // move three steps in the south direction, from (1,16) to (2,16)

    {2}, // pen down
```

```
603     {5,1}, // draw from (3,16) to (3,16)
604     {3}, // trun right, from south to west
605     {5,12}, // draw horizontal line from (3,15) to (3,4)
606
607     // draw left upper vertical line of 5, from (4,4) to (4,9)
608     {4}, // turn left
609     {5,6}, // draw vertical line
610
611     // draw middle horizontal line
612     {4}, // turn left from south to east
613     {5,12}, // from (9,5) to (9,16)
614
615     // draw right lower vertical line
616     {3}, // turn right from east to south
617     {5,6}, // draw 6 steps from (10,16) to (15, 16)
618
619     // draw bottom horizontal line
620     {3}, // turn right from south to west
621     {5,12}, // draw 12 steps from (15,15) to (15,4)
622
623     {6} // print
624   },
625
626   //commands6
627   { // facing east in the beginning
628     {1}, // pen up
629     {5,16}, // move from (0,1) to (0,16)
630     {3}, // turn right from east to south
631     {5,2}, // move three steps in the south direction, from (1,16) to (2,16)
632
633     {2}, // pen down
634     {5,1}, // draw from (3,16) to (3,16)
635     {3}, // trun right, from south to west
636     {5,12}, // draw horizontal line from (3,15) to (3,4)
637
638     // draw left upper vertical line of 5, from (4,4) to (4,15)
639     {4}, // turn left, from west to south
640     {5,12}, // draw vertical line
641
642     // draw bottom horizontal line
643     {4}, // turn left, from south to east
644     {5,12}, // draw 12 steps from (15,5) to (15,16)
645
646     // draw right lower vertical line
647     {4}, // turn left from east to north
648     {5,6}, // draw 6 steps from (16,14) to (16, 9)
```

```
649
650     // draw middle horizontal line
651     {4}, // turn left from north to west
652     {5,12}, // from (9,5) to (9,16)
653
654     // print
655     {6}
656   },
657
658   //commands7
659   { {1}, // pen up
660     {5,3},// move east 3 steps
661     {3}, // turn right from east, facing south
662     {5,3},// move south 3 steps
663     {4}, // turn left from south, face east again
664
665     //## draw top horizontal line
666     {2}, // pen down
667     {5,13},// move 13 steps, from (3,4) to (3,16),
668
669     {3}, // turn right, from east to south
670     {5,12}, // move 12 steps, from (4,16) to (15,16)
671
672     {6} //print out
673   },
674
675   //commands8
676   { {1},  // pen up, facing east by default
677     {5,4}, // move east 4 steps from (0,1) to (0,4)
678     {3},  // turn right, from east to south
679     {5,3}, // move 3 steps, from (1,4) to (3,4)
680     {4},  // turn left, from south to east again
681
682     {2},  // pen down
683
684     // draw top horizontal line
685     {5,12},// move 12 steps, from
686
687     // draw right vertical line, facing south
688     {3},{5,12},
689
690     // draw bottom horizontal line, facing west
691     {3},{5,12},
692
693     // draw left vertical line, facing north
694     {3},{5,12},
```

```
695
696     // pen up, turn south
697     {1}, // pen up
698     {3}, // turn right, from south to east
699     {3}, // turn right, from east to north
700     {5,6}, // move 6 steps, at (9,4)
701     {4}, // turn left, from north to east
702
703     // draw middle line
704     {2},
705     {5,12},
706
707     // print
708     {6}
709   },
710
711   //commands9
712   { {1},  // pen up, facing east by default
713     {5,4}, // move east 4 steps from (0,1) to (0,4)
714     {3},  // turn right, from east to south
715     {5,3}, // move 3 steps, from (1,4) to (3,4)
716     {4},  // turn left, from south to east again
717
718     {2},  // pen down
719
720     // draw top horizontal line
721     {5,12},// move 12 steps, from
722
723     // draw right vertical line, facing south
724     {3},{5,12},
725
726     // draw bottom horizontal line, facing west
727     {3},{5,12},
728
729     // draw left top vertical line, facing north
730     {3},
731     {1},
732     {5,5}, // move to (10,4)
733     {2},
734     {5,7}, // move to (3,4)
735
736     // pen up, turn south
737     {1}, // pen up
738     {3}, // turn right, from south to east
739     {3}, // turn right, from east to north
740     {5,6}, // move 6 steps, at (9,4)
```

```
    {4}, // turn left, from north to east

    // draw middle line
    {2},
    {5,12},

    // print
    {6}
  },
};

std::vector<std::vector<int>> CMDS_MINUS = {
  {1},       // pen up
  {3},{5,9}, // turn right -> SOUTH, move 9 -> (9,0)
  {4},{5,4}, // turn left -> EAST, move 4 -> (9,4)
  {2},{5,10}, // pen down, draw east 10 -> row9 cols 5-14
  {1},{6}
};

void display(int num);
void display(const std::string& str);

int main() {
  std::cout << "Enter an integer: ";
  int num;
  std::cin >> num;
  std::cout << "integer shape is follows:\n"; //this line is needed

  // TODO: Call display(num).

  // Discard the newline left in the buffer after reading the integer,
  // so that the subsequent getline call reads correctly.
  std::cin.ignore(INT_MAX, '\n');
  std::cout << "Enter a string (only supports capital letters, spaces, and digits: ";
  std::string str;
  getline(std::cin, str);
  std::cout << "string shape is follows:\n"; //this line is needed

  // TODO: Call display(str).

  return 0;
}

void display(int num) {
  // TODO:
  // (1) Declare sign as an int initialized to 1.
```

```
787   // (2) If num is negative:
788   //     (a) Set sign to -1.
789   //     (b) Set num to its absolute value (e.g., -123 becomes 123)
790   //         so that num % 10 always returns a non-negative digit.
791
792   // TODO: Declare a TurtleGraphics object named tg.
793
794   // Declare shapes as a vector of Floor objects.
795   std::vector<Floor> shapes;
796   int digit;
797   do {
798     // TODO: Extract the least significant digit of num into variable digit.
799
800     // TODO: Call tg.run() with CMDS_DIGITS[digit] to draw that digit.
801     // Example: if digit is 3, run CMDS_DIGITS[3], which draws the digit 3.
802
803     // TODO: Call tg.getFloor() and append the result to shapes.
804
805     // TODO: Call tg.restart() to clear the floor and reset the turtle
806     //       to position (0, 0) facing east, ready to draw the next digit.
807
808     // TODO: Remove the least significant digit from num.
809     //       Example: if num is 123, it becomes 12.
810
811   } while (...); // TODO: Continue as long as num has remaining digits.
812
813   // TODO: If sign is -1, call tg.run() with CMDS_MINUS to draw the minus sign,
814   //       then append tg.getFloor() to shapes.
815   //
816   // Note: push_back() adds to the end, so digits are stored in reverse order
817   // (least significant first). Example:
818   //   display(-123): shapes contains: [3, 2, 1, -]
819   //   display(123): shapes contains: [3, 2, 1]
820
821   // Get the first element of shapes to determine row/column dimensions.
822   // shapes is guaranteed non-empty since num has at least one digit.
823   const Floor& floor = ...; // TODO: first element of shapes
824
825   // TODO: Outer loop: iterate over each row of the floor.
826   for (int row = 0; row < ...; row++) {
827
828     // TODO: Middle loop: iterate through shapes in REVERSE order (last index to first).
829     //   Reason: Digits were stored least-significant-first.
830     //   Reversing the order ensures they print from left to right (most significant
        digit first).
831     for (int k = ...; ...; ...) {
```

```
832
833       // TODO: Inner loop: iterate over each column of the current shape.
834       for (int col = ...; col < ...; ...) {
835         std::cout << shapes[...].at(..., ...); // TODO: print character at (row, col) of
              shapes[k]
836       }
837     }
838
839     std::cout << '\n';
840   }
841 }
842
843 void display(const std::string& str) {
844   TurtleGraphics tg;
845   std::vector<Floor> shapes;
846
847   // TODO: Process each character ch in str:
848   // (1) Uppercase letter (isupper): run CMDS_CAP_LETTERS[ch - 'A'].
849   //     Example: 'A' -> CMDS_CAP_LETTERS[0], 'B' -> CMDS_CAP_LETTERS[1].
850   // (2) Lowercase letter (islower): convert to uppercase with toupper(ch),
851   //     then apply the same logic as (1).
852   // (3) Digit character (isdigit): run CMDS_DIGITS[ch - '0'].
853   //     Example: '0' -> CMDS_DIGITS[0], '1' -> CMDS_DIGITS[1].
854   //     Note: unlike display(int), where digits are extracted numerically,
855   //     here a digit is a char (from '0' to '9'), so use ch - '0' to get its index.
856   // (4) Space character (isspace): call tg.restart() to produce a blank floor.
857   // (5) Any other character: skip it with continue.
858   // (6) After processing ch, append tg.getFloor() to shapes.
859   // (7) Call tg.restart() to reset the floor and turtle for the next character.
860
861   if (...) { // TODO: condition: shapes is empty (str had no drawable characters)
862     std::cout << "no string inputed" << "\n";
863     return;
864   }
865
866   // Shapes are stored left-to-right, matching the order of characters in str.
867   // Example: "AB 12" -> shapes contains: [A, B, space, 1, 2]
868   const Floor& floor = ...; // TODO: first element of shapes
869
870   // TODO: Outer loop: iterate over each row of the floor.
871   for (int row = ...; row < ...; ...) {
872
873     // TODO: Middle loop: iterate over shapes in FORWARD order (first to last),
874     //       since characters in str are processed left to right.
875     for (int k = ...; ...; ...) {
876
```

```
      // TODO: Inner loop: iterate over each column of the current shape.
      for (int col = ...; col < ...; ...) {
        std::cout << shapes[...].at(..., ...); // TODO: print character at (row, col) of
            shapes[k]
      }
    }

    std::cout << '\n';
  }
}
```

7.4 Compile and Run

Compile all files together with the following commands.

```
g++ -o draw Floor.cpp Turtle.cpp TurtleGraphics.cpp drawSideBySide.cpp
./draw
```

7.5 Sample Input and Output

If we input -21 and "CS 135" (without quotes) respeectively, here are a sample output (I omit the output of each individual character).

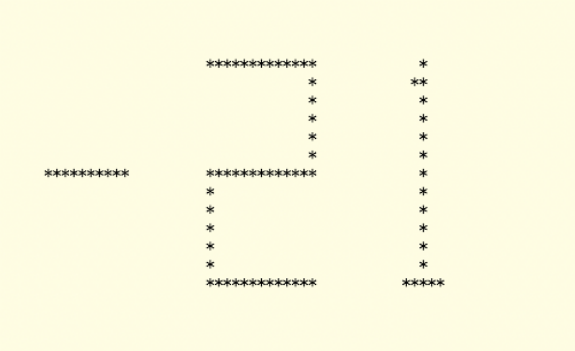


Figure 1: Display -21

7.6 Submission for Task D

Submit `drawSideBySide.cpp` to gradescope.

8 Makefile: a Tool for Project Build Management

Compiling and linking multiple source files into an executable can be time-consuming and error-prone. Using a Makefile simplifies this process.

Download link: **Makefile**. Its contents are as follows.

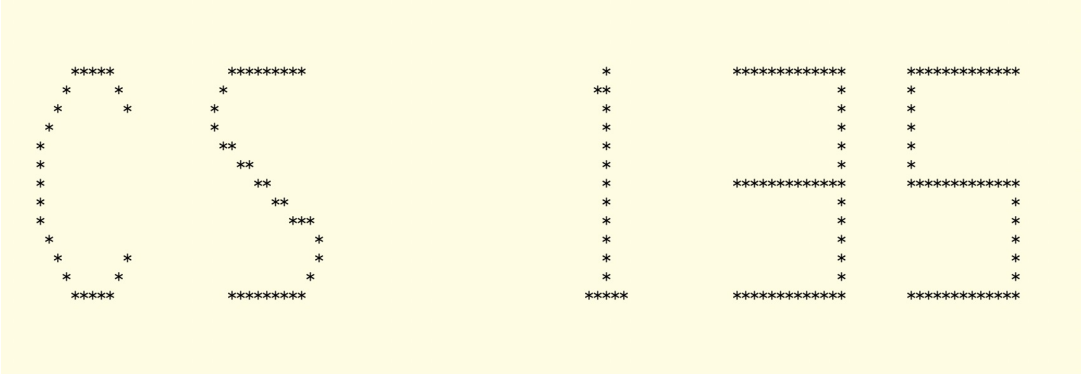


Figure 2: Display “CS 135”

```
# Compiler
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -pedantic

# Object files
FLOOR_OBJ = Floor.o
TURTLE_OBJ = Turtle.o
TG_OBJ = TurtleGraphics.o

# -----------------------
# Default target
# -----------------------
all: floor turtle tg draw

# -----------------------
# Task A: Floor only
# -----------------------
floor: FloorUnitTest.o $(FLOOR_OBJ)
    $(CXX) $(CXXFLAGS) -o floor FloorUnitTest.o $(FLOOR_OBJ)

# -----------------------
# Task B: Turtle only
# -----------------------
turtle: TurtleUnitTest.o $(TURTLE_OBJ)
    $(CXX) $(CXXFLAGS) -o turtle TurtleUnitTest.o $(TURTLE_OBJ)

# -----------------------
# Task C: TurtleGraphics
# -----------------------
tg: TurtleGraphicsUnitTest.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
    $(CXX) $(CXXFLAGS) -o tg TurtleGraphicsUnitTest.o $(TG_OBJ) $(TURTLE_OBJ) $(
       FLOOR_OBJ)
```

```
32
33 # ----------------------
34 # Draw program (depends on ALL)
35 # ----------------------
36 draw: drawSideBySide.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
37     $(CXX) $(CXXFLAGS) -o draw drawSideBySide.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
38
39 # ----------------------
40 # Compilation rules
41 # ----------------------
42 Floor.o: Floor.cpp Floor.hpp
43     $(CXX) $(CXXFLAGS) -c Floor.cpp
44
45 Turtle.o: Turtle.cpp Turtle.hpp
46     $(CXX) $(CXXFLAGS) -c Turtle.cpp
47
48 TurtleGraphics.o: TurtleGraphics.cpp TurtleGraphics.hpp Turtle.hpp Floor.hpp
49     $(CXX) $(CXXFLAGS) -c TurtleGraphics.cpp
50
51 # Unit tests
52 FloorUnitTest.o: FloorUnitTest.cpp Floor.hpp
53     $(CXX) $(CXXFLAGS) -c FloorUnitTest.cpp
54
55 TurtleUnitTest.o: TurtleUnitTest.cpp Turtle.hpp
56     $(CXX) $(CXXFLAGS) -c TurtleUnitTest.cpp
57
58 TurtleGraphicsUnitTest.o: TurtleGraphicsUnitTest.cpp TurtleGraphics.hpp Turtle.hpp
      Floor.hpp
59     $(CXX) $(CXXFLAGS) -c TurtleGraphicsUnitTest.cpp
60
61 # Draw program
62 drawSideBySide.o: drawSideBySide.cpp TurtleGraphics.hpp Turtle.hpp Floor.hpp
63     $(CXX) $(CXXFLAGS) -c drawSideBySide.cpp
64
65 # ----------------------
66 # Clean
67 # ----------------------
68 clean:
69     rm -f *.o floor turtle tg draw
```

To compile and link Floor.cpp and FloorUnitTest.cpp in Task A, run

```
make floor
```

Then test the default constructor Floor class using

```
./floor A
```

To compile and link Turtle.cpp and TurtleUnitTest.cpp in Task B, run

```
make turtle
```

Test reset method of Turtle class using

```
./turtle A
```

To compile and link Floor.cpp, Turtle.cpp, TurtleGraphics.cpp, and TurtleGraphicsUnitTest.cpp in Task C, run

```
make tg
```

Then test default constructor of TurtleGraphics class using

```
./tg A
```

To compile and link Floor.cpp, Turtle.cpp, TurtleGraphics.cpp, and drawSideBySide.cpp in Task D, run

```
make draw
```

or simply

```
make
```

Then run the executable file using

```
./draw
```

Call the following command to clean all generated object files and executable files.

```
make clean
```